

Krkn-operator

Version 1.0.0, 2026-08-18

Home

Introduction

Course Title: Krkn-operator

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Introduction to Krkn-operator
- Installation
- Configuration and Management
- Lifecycle Management
- Hands-on Lab
- Introduction to Krkn-operator
- Installation
- Configuration and Management
- Introduction to Krkn-operator
- Installation
- Configuration and Management
- Lifecycle Management
- Hands-on Lab

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduction to Krkn-operator

Introduction to Krkn-operator

The Krkn-operator is a Kubernetes-native platform designed to streamline Chaos Engineering by centralizing the deployment, configuration, and execution of experiments. It moves beyond manual scenario execution by providing an integrated web interface for comprehensive cluster and chaos orchestration.

Key features and capabilities include:

- **Centralized Orchestration:** Manage chaos experiments, user permissions, and cluster configurations from a single, unified web console.
- **Kubernetes-Native Architecture:** Built to operate directly within Kubernetes environments, leveraging native resources for seamless integration.
- **Chaos Management:** Simplifies the lifecycle of chaos engineering, from initial deployment to the execution and monitoring of complex experiments.
- **Operational Efficiency:** Eliminates the need for manual command-line execution by offering a robust management platform for chaos scenarios.# Introduction to Krkn-operator

Introduction to Krkn-operator

The Krkn-operator is a Kubernetes-native platform designed to streamline and orchestrate Chaos Engineering experiments. By centralizing management within a web interface, it eliminates the need for manual execution of chaos scenarios.

- **Chaos Engineering Orchestration:** Provides a unified platform to deploy, configure, and manage chaos experiments directly within Kubernetes.
- **Centralized Management:** Simplifies administrative tasks, including the management of users, clusters, permissions, and experiment execution workflows.
- **Operational Efficiency:** Transitions chaos testing from manual, ad-hoc execution to a structured, platform-driven approach.# Introduction to Krkn-operator

Introduction to Krkn-operator

The Krkn-operator is a Kubernetes-native platform designed to streamline Chaos Engineering by providing a centralized web interface for experiment orchestration. It replaces manual scenario execution with a unified management system for clusters, user permissions, and chaos workflows.

Key features include:

- **Chaos Orchestration:** Simplifies the deployment, configuration, and execution of chaos experiments directly within the cluster.
- **Centralized Management:** Provides a single web console to manage users and coordinate complex chaos scenarios.

- **Kubernetes-native Architecture:** Built to operate seamlessly within the Kubernetes ecosystem, ensuring deep integration with cluster resources.
- **Operational Efficiency:** Eliminates the need for manual script management, enabling scalable and repeatable fault-injection testing.

Overview of Kubernetes-native Chaos Engineering

Overview of Kubernetes-native Chaos Engineering

Kubernetes-native Chaos Engineering is the practice of proactively injecting controlled failures into a Kubernetes environment to identify weaknesses in system resiliency. By systematically disrupting components—such as killing pods, inducing network latency, or exhausting resources—you can observe how your applications and infrastructure react under stress.

The primary goal is to shift from reactive firefighting to proactive design, ensuring that your systems can gracefully recover from failures before they impact end-users.

The Role of Krkn-operator

In the context of the Kraken (Krkn) ecosystem, the **Krkn-operator** serves as the control plane for orchestrating these experiments within a Kubernetes cluster. Unlike manual script execution, the Krkn-operator provides a structured, automated way to lifecycle-manage chaos experiments.

Core Pillars of Krkn Chaos Engineering

- **Observability-Driven:** Integration with PromQL-based metrics allows you to measure the impact of chaos in real-time, providing a "Resiliency Score" for your applications.
- **Automation:** By moving away from manual `kubectl` commands, the operator allows for the definition of reusable, version-controlled chaos scenarios.
- **Safety and Control:** The architecture is designed to allow granular permission management, ensuring that chaos experiments are scoped strictly to intended targets.
- **Multi-Cluster Orchestration:** Chaos is rarely isolated to a single node. The Krkn-operator architecture supports executing experiments across multiple clusters, simulating real-world regional outages or large-scale control plane issues.

Why Kubernetes-Native?

Traditional chaos engineering tools often operate at the infrastructure level (e.g., shutting down virtual machines). Kubernetes-native chaos engineering, facilitated by the Krkn-operator, focuses on the orchestration layer:

1. **Abstraction of Complexity:** Through tools like `krknctl` and the operator-based web interface, users do not need to manage the underlying container runtime complexities.
2. **Kubernetes Resource Awareness:** Experiments understand Kubernetes primitives. For example, a scenario can target a specific `Deployment` or `StatefulSet` rather than just a random IP address, ensuring the experiment maintains semantic relevance to your application architecture.

3. **CI/CD Integration:** Because chaos scenarios are defined as reproducible configurations (or container images within the `krkn-hub`), they can be treated as first-class citizens in a CI/CD pipeline, allowing developers to test resiliency as part of the standard deployment process.

Architectural Components

- **Krkn-hub:** A repository of pre-built container images containing specific chaos scenarios.
- **Krkn-operator:** The controller that watches for Chaos Custom Resources (CRDs) within your cluster, triggering the execution of scenarios from the hub.
- **Chaos Studio:** A management layer that enables the design of complex workflows (serial or parallel execution) and monitors the health of the target cluster during the experiment.

By leveraging the Krkn-operator, teams can move from simply "knowing" their system is fragile to actively validating their auto-scaling policies, pod disruption budgets, and failover logic in a safe, controlled manner.

Overview of Kubernetes-native Chaos Engineering

Overview of Kubernetes-native Chaos Engineering

Kubernetes-native chaos engineering is the practice of proactively injecting controlled failures into a cluster environment to identify weaknesses in system resiliency, recoverability, and observability. By treating the cluster as a dynamic and inherently unstable environment, organizations can move beyond theoretical stability to empirical proof of system durability.

The Core Philosophy

In a distributed environment like Kubernetes, failures—such as pod evictions, node crashes, or network latency—are inevitable. Kubernetes-native chaos engineering shifts the perspective from "preventing failure" to "building for failure."

By integrating chaos experiments directly into the Kubernetes lifecycle, we ensure that:

- * **System observability is verified:** We confirm that our monitoring and alerting tools actually trigger when anomalies occur.
- * **Resiliency is quantifiable:** Through tools like Krkn, we can assign a "Resiliency Score" based on PromQL-based metrics to measure how well an application sustains load during a disruption.
- * **Automation is embedded:** Chaos is not a one-time audit but a continuous part of the CI/CD pipeline, ensuring that new deployments do not regress the system's baseline reliability.

Why Krkn-operator?

While many chaos tools exist, **Krkn-operator** simplifies the orchestration of complex chaos scenarios. Traditional chaos engineering often requires managing disparate configuration files and complex environment variables. The Krkn ecosystem, specifically the operator, provides:

- **Abstraction of Complexity:** It removes the burden of managing individual container images or manual `podman/docker` commands, allowing engineers to focus on the **intent** of the experiment.
- **Unified Control Plane:** It provides a centralized interface for multi-cluster execution, allowing you to blast chaos across global environments simultaneously.
- **Standardized Scenarios:** Utilizing `krkn-hub`, users gain access to a library of pre-defined, battle-tested scenarios that cover everything from network partition simulation to resource

exhaustion (CPU/Memory).

How Krkn Fits into the Ecosystem

The Krkn ecosystem is designed to be modular, ensuring that users can engage with chaos engineering at their own comfort level:

- **Krknctl:** The recommended CLI tool for rapid prototyping and local execution, abstracting the runtime environment.
- **Krkn-operator:** The platform-level engine that manages the full lifecycle of chaos experiments within the cluster, including user management, workspace isolation, and visual workflow design in "Chaos Studio."
- **Krkn-hub:** The source of truth for scenario definitions, providing the building blocks for every experiment.

By adopting a Kubernetes-native approach, the Krkn-operator treats "chaos" as a first-class citizen of the cluster—governed by the same RBAC, namespaces, and observability standards as your production applications. This integration ensures that chaos experiments are not just "attacks" on the system, but rather structured, measurable engineering tests.[# Overview of Kubernetes-native Chaos Engineering](#)

Overview of Kubernetes-native Chaos Engineering

Kubernetes-native Chaos Engineering is the practice of proactively injecting controlled failures into a Kubernetes environment to identify and mitigate weaknesses before they impact end-users. By intentionally inducing faults—such as pod terminations, network latency, or resource exhaustion—teams can validate the resiliency of their applications and the self-healing capabilities of the underlying cluster.

Why Chaos Engineering Matters

In distributed systems, failures are inevitable. Infrastructure components, network partitions, and application bugs are common occurrences. Relying solely on manual testing is insufficient because it often fails to account for the complex, emergent behaviors that arise in highly dynamic environments like Kubernetes.

Chaos engineering shifts the focus from "testing for success" to "testing for failure," allowing SREs and developers to:

- * **Validate Auto-scaling:** Ensure your cluster scales correctly when a node is suddenly terminated.
- * **Verify Health Checks:** Confirm that liveness and readiness probes accurately detect and recover from unstable application states.
- * **Measure Mean Time to Recovery (MTTR):** Observe how quickly your system restores service after a failure occurs.
- * **Build Confidence:** Shift from reactive troubleshooting to proactive system hardening.

The Krkn Approach

Krkn is designed to be Kubernetes-native, meaning it leverages standard Kubernetes primitives to orchestrate chaos. Unlike legacy tools that require custom agents or complex sidecar injection, Krkn treats chaos scenarios as first-class citizens within the cluster ecosystem.

The Krkn ecosystem consists of several components that simplify this workflow:

- **Krkn-hub:** A repository of pre-packaged, containerized chaos scenarios. These scenarios are designed to be run as standard Kubernetes jobs, allowing you to use existing CI/CD pipelines and infrastructure tools to trigger tests.
- **krknctl:** An orchestration layer that abstracts the complexity of manual configuration. It manages the lifecycle of these chaos jobs, allowing users to focus on defining the blast radius and success criteria rather than runtime management.
- **Chaos Studio:** Provides a visual interface to manage workflows, enabling users to chain multiple scenarios together in serial or parallel sequences to simulate real-world, cascading failure scenarios.

Core Concepts

Understanding the following concepts is essential for effective chaos orchestration:

- **Blast Radius:** The scope of the cluster impacted by the experiment. In Krkn, this can be limited to specific namespaces, labels, or nodes.
- **Steady State:** The measurable baseline behavior of your system under normal conditions. Before injecting chaos, you must define what "normal" looks like (e.g., latency under 200ms, 99.9% success rate).
- **Observation:** Using tools like PromQL to monitor the system throughout the experiment. Krkn integrates with Prometheus to measure the "Resiliency Score," quantifying how well your application maintains its steady state during a fault injection.

By embedding these practices directly into your Kubernetes workflow, you transition from simply deploying applications to engineering resilient, self-healing platforms.

Architecture and core components

Architecture and core components

The Krkn-operator is designed as a Kubernetes-native controller, leveraging the Operator pattern to manage the lifecycle of chaos engineering experiments directly within the cluster. By utilizing custom resource definitions (CRDs) and a control loop, it provides a declarative way to orchestrate complex failure scenarios.

Architectural Overview

The architecture follows the standard Kubernetes controller pattern, ensuring that the desired state of chaos experiments is continuously reconciled with the actual state of the cluster.

```
graph TD
  A[User/Admin] -->|Apply CRD| B[Kubernetes API Server]
  B --> C[Krkn-Operator Controller]
  C -->|Orchestrates| D[Chaos Experiments]
  D -->|Injects Faults| E[Target Workloads/Nodes]
```

```
C -->|Monitors| F[Cluster Health/Metrics]
F -->|Feedback Loop| C
```

Core Components

The functionality of the Krkn-operator is powered by several internal building blocks:

- **Controller Manager:** The central brain of the operator. It watches for changes to Krkn-specific custom resources. When a new chaos scenario is defined, the controller triggers the execution logic.
- **Chaos Engine:** The execution engine responsible for the lifecycle of an experiment. It handles the injection of faults (e.g., CPU/Memory hogs, namespace deletions) and ensures that the system returns to its original state after the experiment duration expires.
- **Reconciliation Loop:** This component constantly monitors the health of the target environment. It ensures that if a component fails—whether due to infrastructure issues, application bugs, or dependencies—the system recognizes the failure immediately to avoid extended downtime.
- **Target Provider Interface:** An abstraction layer that allows the operator to interface with different infrastructure environments, such as standalone Kubernetes clusters or clusters managed via Advanced Cluster Management (ACM).

Design Principles

To ensure reliability during chaos injection, the operator is built on these foundational principles:

Principle	Description
	High Availability
The operator is designed for HA deployment, ensuring that the control plane for chaos engineering remains resilient even when the underlying infrastructure is under stress.	
Minimal Recovery Time	The operator logic prioritizes fast recovery. Once an experiment is concluded, the controller initiates recovery logic to restore normal operations, preventing "chaos residue."
	Declarative Configuration

Chaos scenarios are defined as Kubernetes objects. This allows teams to store, version, and audit chaos experiments using standard GitOps workflows.	
Observability Integration	The operator integrates with monitoring systems to verify that the system detects failures as soon as they are injected, validating the effectiveness of existing alerts and self-healing mechanisms.

Component Responsibilities

The separation of concerns within the Krkn-operator architecture allows it to handle diverse failure modes—ranging from resource starvation (CPU/Memory Hogs) to systemic disruptions (Namespace Failures or Time Skewing). By offloading the execution to the controller, the operator ensures that even if the primary interface is disconnected, the scheduled experiments or recovery tasks remain active, maintaining control over the chaos.

Architecture and core components

The **Krkn-operator** is designed as a Kubernetes-native controller that adheres to the Operator pattern, providing a lifecycle management framework for chaos engineering. By leveraging custom resource definitions (CRDs) and reconciliation loops, it allows operators to manage complex failure scenarios with the same declarative approach used for managing applications in Kubernetes.

Core Architectural Principles

The architecture of the Krkn-operator is built upon three fundamental pillars:

- **Kubernetes-Native Design:** The operator runs within the cluster and watches for changes to custom resources. This ensures that the state of your chaos experiments is stored within the etcd database, allowing for automated orchestration and state reconciliation.
- **High Availability (HA):** To ensure reliability during testing, the operator is designed for HA deployment. This minimizes initialization time and ensures that the controller can recover quickly if the infrastructure or the underlying service it monitors experiences a failure.
- **Decoupled Orchestration:** By separating the management of chaos experiments (the "control plane") from the execution of the fault (the "data plane"), the operator can manage multiple target clusters, even across different infrastructures or providers like ACM (Advanced Cluster Management).

Primary Components

The Krkn-operator ecosystem consists of several specialized components that interact to provide a seamless chaos engineering experience:

1. The Controller (Reconciliation Loop)

The core engine of the operator. It continuously monitors the state of **ChaosScenario** custom resources. When a change is detected, it triggers the necessary logic to initiate, monitor, or stop chaos experiments. It is optimized for "fast failure recognition," ensuring that if a chaos experiment fails to launch or encounters an error, the system is notified immediately to prevent extended downtime.

2. Chaos Experiment Engine

This is the execution layer that carries out the specific disruptions defined in the configuration. The engine is responsible for the actual "heavy lifting," such as:

- * **Resource Stressors:** Managing CPU and Memory consumption (e.g., CPU/Memory Hogs).
- * **Infrastructure Manipulation:** Executing time-skewing operations or simulating namespace failures.
- * **Validation Logic:** Verifying that systems remain resilient, such as ensuring that pods reschedule correctly after a deletion or that system time is reset accurately once an experiment concludes.

3. Admin & User Interface (API/Web)

The operator provides an interface for administrative and user interactions.

- * **Admin Layer:** Provides full platform access, allowing for the registration of target clusters, management of user groups, and configuration of private registries.
- * **Target Provider Integration:** The architecture supports integration with external providers (such as ACM) to extend chaos orchestration beyond a single cluster, allowing admins to map chaos scenarios to specific environment topologies.

4. Custom Resource Definitions (CRDs)

These represent the "manifests" of chaos. By defining experiments in YAML, users maintain a declarative record of what was tested, on which cluster, and for what duration. This enables reproducibility in testing, which is critical for verifying fixes after a system failure.



The controller is designed to recognize failures as soon as possible. Because chaos engineering involves intentional disruption, the operator ensures that even if an underlying infrastructure or service fails during an experiment, the system attempts to maintain stability through fast recovery logic and minimal component initialization time.

Architectural Workflow

1. **Definition:** An administrator or user defines a **ChaosScenario** (e.g., a Memory Hog) via the web interface or CLI.
2. **Orchestration:** The Krkn-operator detects the new resource and schedules the experiment.
3. **Execution:** The engine injects the fault into the target cluster.
4. **Observation:** The controller monitors the health of the target components.
5. **Reconciliation:** Once the experiment duration concludes, the operator verifies the cleanup process—such as verifying the return of normal CPU/Memory usage or verifying that system time has been properly synchronized.

Architecture and core components

The `krkn-operator` acts as the control plane for chaos engineering within a Kubernetes ecosystem. By leveraging the operator pattern, it reconciles the desired state of chaos experiments with the actual state of the cluster, ensuring that fault injection is orchestrated reliably and safely.

Architectural Overview

The architecture of the `krkn-operator` is designed to be Kubernetes-native, meaning it treats chaos scenarios as custom resources. This approach allows users to manage chaos experiments using standard `kubectl` commands, GitOps workflows, and integration with existing CI/CD pipelines.

The architecture comprises three primary layers:

1. The Controller Layer (Control Plane)

The Controller is the heart of the `krkn-operator`. It runs as a pod within your cluster and continuously watches for `ChaosExperiment` custom resources. * **Reconciliation Loop:** Upon detecting a new experiment, the controller validates the configuration and initiates the workflow. * **State Management:** It manages the lifecycle of the experiment, from initialization and execution to post-experiment cleanup and recovery. * **Fast Recovery Logic:** Designed for minimal initialization and recovery times, the controller ensures that once an experiment concludes, the infrastructure is returned to its steady state immediately to avoid extended downtime.

2. The Execution Layer (Target Clusters)

This layer handles the actual fault injection. The operator supports a multi-cluster architecture where it can interface with the local cluster or remote target clusters. * **Target Providers:** The operator integrates with infrastructure managers (such as ACM - Advanced Cluster Management) to provision and target specific clusters for chaos scenarios. * **Execution Hooks:** These are the triggers that cause the faults (e.g., CPU Hog, Memory Hog, or Namespace deletion).

3. The Management Interface

This layer provides the entry point for administrators and users to interact with the operator. * **Role-Based Access Control (RBAC):** Integrates with Kubernetes native RBAC to define who can create, view, or manage chaos scenarios. * **User/Group Management:** Administrators define cluster access levels, allowing for segregated testing environments where specific teams can only influence designated clusters or namespaces.

Core Components

To understand how the `krkn-operator` functions, it is essential to identify the following components:

Component	Function
ChaosExperiment (CRD)	The custom resource definition that defines the parameters of the chaos scenario, such as target labels, duration, and the type of fault (e.g., CPU Hog).

Component	Function
Controller Manager	The process responsible for watching the CRDs and executing the logic required to induce the failure.
Target Provider	The connector that bridges the krkn-operator to the specific infrastructure where the fault should be injected.
Registry Manager	Handles the visibility and configuration of container registries required to pull the chaos experiment images.

Chaos Execution Engine

The engine is responsible for the specific types of stress tests identified in the lifecycle. The architecture ensures that these components are decoupled:

- **Resource Stressors:** Components responsible for injecting high load (CPU/Memory Hogs).
- **Environmental Manipulators:** Components that perform system-level changes, such as Time Skewing (manipulating system clocks) or Namespace Failures (deleting namespaces).

By separating the **Control Plane** (Controller) from the **Data Plane** (Execution/Stressors), the **krkn-operator** maintains High Availability (HA). Even if a specific experiment causes localized disruption to a target application, the operator remains functional, ready to trigger recovery logic or abort the process if thresholds are exceeded.

Use cases for chaos orchestration

Use cases for chaos orchestration

Chaos orchestration is the systematic practice of injecting controlled failures into a system to verify its resilience against the inherent uncertainties of distributed environments. By leveraging the **Krkn-operator**, teams can move beyond manual testing and transition into automated, repeatable resilience validation.

Why Chaos Orchestration Matters

Distributed systems often fail due to assumptions that rarely hold true in production, such as the reliability of the network or the infinite availability of shared resources. Chaos orchestration addresses these by proactively challenging the system's design.

Key Use Cases

Through the Krkn-operator, you can implement the following strategic use cases:

Use Case	Description
Validation of High Availability (HA)	Simulate the loss of individual nodes or pods to ensure that orchestration layers (like Kubernetes) correctly reschedule workloads and that service traffic reroutes without user-facing downtime.

Use Case	Description
Dependency Resilience Testing	Inject network latency or packet loss between microservices to evaluate how application code handles timeouts and circuit breaking. This ensures that a degradation in one component does not trigger a cascading failure across the entire system.
Resource Exhaustion Profiling	Artificially induce memory or CPU spikes on specific deployments. This helps verify that Horizontal Pod Autoscalers (HPA) trigger correctly and that the application behaves gracefully under load rather than crashing unexpectedly.
Resiliency Score Benchmarking	Use the Krkn-operator to measure your system against a "Steady State." By defining critical SLOs (e.g., API server latency), you can quantify how much your system's performance degrades during an experiment, providing a measurable Resiliency Score .
Automated Compliance Verification	Integrate chaos experiments into your CI/CD pipelines to ensure that every release maintains the required reliability standards before being promoted to production.

Mapping Chaos to Business Goals

Chaos orchestration is not just about breaking things; it is about validating the **corrective** mechanisms built into your architecture.

- **Defining the Steady State:** Before orchestration begins, you must establish the "normal" behavioral baseline. Without this, you cannot objectively measure the impact of an experiment.
- **Analyzing Metrics:** During orchestration, the Krkn-operator allows you to monitor key KPIs. By comparing these against your established steady state, you can identify "blind spots" where monitoring or alerting might be insufficient.
- **Continuous Improvement:** Use the data gathered from orchestrated experiments to prioritize architectural changes. If an experiment proves a service is not resilient, the output provides the specific data needed to justify remediation efforts to stakeholders.



For complex environments, use **Chaos Studio**—a core feature of the Krkn ecosystem—to build visual workflows. This allows you to design scenarios that execute in series or parallel, enabling you to simulate multi-stage failure modes, such as a node failure occurring simultaneously with a network partition. ====#
Use cases for chaos orchestration

= Use cases for chaos orchestration

Chaos orchestration is the practice of systematically planning, executing, and automating the injection of faults into a distributed system to validate its resilience. By using tools like the **krkn-operator**, teams move beyond manual, ad-hoc testing toward a continuous, integrated resilience strategy.

== Why Orchestrate Chaos?

In complex Kubernetes environments, individual components often behave correctly in isolation, but fail under the pressure of interconnected dependencies. Chaos orchestration addresses this by enabling teams to move from reactive troubleshooting to proactive resilience modeling.

=== Key Use Cases

Use Case	Description
Validation of SLOs/SLIs	Verify that Service Level Objectives (SLOs) remain intact during component failure. Orchestrating chaos helps determine if automated recovery mechanisms (e.g., auto-scaling, self-healing pods) trigger within the required timeframes.
Resilience Score Benchmarking	Measure the "Resiliency Score" of an application by observing how metrics (such as API latency or request success rates) fluctuate during orchestrated fault injection.
Dependency Failure Analysis	Test how an application handles the loss of shared resources, such as database connectivity, service-to-service communication timeouts, or localized network partitions.
Recovery Time Objective (RTO) Testing	Quantify the time required for a service to return to its steady state after a synthetic failure, ensuring compliance with business continuity requirements.
Drift and Configuration Validation	Confirm that infrastructure-as-code (IaC) configurations and Kubernetes resource limits actually perform as intended when the system is under duress.

== Chaos Workflows: Serial vs. Parallel Execution

A core benefit of chaos orchestration within the Krkn ecosystem is the ability to design **Visual Chaos Workflows**. These workflows are not merely single-command executions; they are structured experiments:

- **Serial Execution:** Used for validating cascading failures. For example, testing the impact of a database outage followed by a secondary network latency spike to see if the recovery process for the first failure is interrupted by the second.
- **Parallel Execution:** Used to simulate "stress" scenarios or multi-vector attacks, such as injecting high CPU utilization across multiple worker nodes simultaneously to observe the behavior of the Kubernetes scheduler and cluster-autoscaler under collective pressure.

== Integrating Chaos into the Lifecycle

Orchestrating chaos is most effective when it is integrated into the software development lifecycle:

1. **Steady State Definition:** Before orchestrating, you must define the "steady state." This involves baseline performance metrics (e.g., request latency, error

rates, throughput).

2. **Hypothesis Injection:** Select a component and hypothesize its behavior under failure. For example: **"If I terminate a core service pod, the redundant replica will take over traffic within 5 seconds without user-visible latency spikes."**
3. **Automated Orchestration:** Use the **krkn-operator** to execute the experiment. By orchestrating this, you ensure consistency—the experiment is performed the same way every time it is triggered, allowing for accurate comparison of results over different software versions.
4. **Resilience Improvement:** Analyze the variance between the "steady state" and the "chaos state." If the outcome deviates from the hypothesis, use the data to refactor the application or infrastructure, then re-run the orchestration to verify the fix. # Use cases for chaos orchestration

= Use cases for chaos orchestration

Chaos orchestration is the practice of automating, scheduling, and sequencing chaos experiments to move beyond simple, manual fault injection. By leveraging the **Krkn-operator**, teams can move from ad-hoc testing to continuous resilience verification.

== Why Chaos Orchestration Matters

In a distributed system, individual components often appear healthy, but failures emerge when dependencies interact under stress. Orchestration allows you to model these complex interactions, ensuring your system remains resilient against the "fallacies of distributed computing," such as network latency, partition events, and resource exhaustion.

By using the **Chaos Studio** interface within the operator, you can move from executing isolated tests to building robust scenarios that reflect real-world production incidents.

== Core Use Cases

=== 1. Validating Resilience to Network Instability Distributed systems rely on the assumption that the network is reliable. Chaos orchestration allows you to: * **Simulate Latency:** Gradually inject latency between services to determine the impact on your SLOs. * **Network Partitioning:** Test how your application handles split-brain scenarios or intermittent connectivity between nodes. * **Packet Loss Injection:** Verify if your retry logic and circuit breakers successfully mitigate packet loss before they impact the end user.

=== 2. Testing Resource Saturation and Auto-scaling A common point of failure is when services consume unexpected amounts of resources. Orchestration allows for: * **Resource Exhaustion:** Intentionally spiking CPU or memory usage on specific pods to see if Horizontal Pod Autoscalers (HPA) trigger as expected. * **Dependency Stress:** Measuring how your application behaves when a shared

database or cache resource becomes unavailable or throttled.

=== 3. Measuring Resiliency Score The Krkn-operator provides the ability to calculate a **Resiliency Score**. Orchestration is critical here because: * It allows for the execution of a defined "steady state" measurement before, during, and after the chaos injection. * By orchestrating scenarios, you can compare the current system performance against pre-defined SLOs, providing a quantifiable metric for improvements made to the system's architecture.

=== 4. Automated Recovery and Self-Healing Validation Rather than just breaking things, orchestration is used to verify that the system recovers without manual intervention. * **Scenario Chaining:** You can design a workflow where a pod is killed, and the operator tracks the time taken for the replacement pod to become "Ready" and receive traffic. * **Serial and Parallel Execution:** Create complex dependencies by running experiments in series (e.g., kill a node, then induce network latency) or in parallel to simulate a "cascading failure" scenario.

=== 5. Continuous Resilience in CI/CD Chaos orchestration enables the integration of resilience testing into your deployment pipeline. By using the Krkn-operator's configuration, teams can: * Automatically trigger a set of "sanity check" chaos experiments whenever a new version of a service is deployed. * Use the web interface to visualize these workflows, ensuring that resilience is a shared, visible metric across development and operations teams.

Before orchestrating any scenario, always ensure you have established a clear **steady state**. Without defined metrics for success, you cannot effectively analyze the impact of the injected chaos or iterate on your infrastructure's improvements.

== [Introduction to Krkn-operator](#)

=== [Overview of Kubernetes-native Chaos Engineering](#)

=== [Architecture and core components](#)

=== [Use cases for chaos orchestration](#)

== [Introduction to Krkn-operator](#)

=== [Overview of Kubernetes-native Chaos Engineering](#)

=== [Architecture and core components](#)

=== [Use cases for chaos orchestration](#)

== [Installation](#)

Installation

The `krkn-operator` is deployed on Kubernetes or OpenShift clusters primarily using Helm

charts.

- **Prerequisites:** Ensure your environment meets the necessary cluster requirements before deployment.
- **Deployment Method:** Use Helm to install the operator, ensuring proper namespace management and creation to maintain an organized cluster environment.
- **Configuration:** Optionally attach configuration files via the File Management interface to customize your deployment parameters.# Installation

Installation

The `krkn-operator` is designed for deployment on Kubernetes or OpenShift environments. Installation is managed through Helm charts, ensuring a standardized and repeatable setup process.

- **Prerequisites:** Ensure your cluster meets the necessary requirements before initiating the deployment.
- **Helm Deployment:** Use Helm to deploy the operator directly into your cluster.
- **Namespace Management:** Establish dedicated namespaces to isolate the operator and its related resources.
- **Configuration:** During or after installation, you can mount optional configuration files via the File Management interface to customize your chaos engineering environment.# Installation

Installation

The Krkn-operator is designed for deployment on Kubernetes or OpenShift environments using Helm charts.

Key installation considerations include:

- **Prerequisites:** Ensure your Kubernetes or OpenShift cluster meets the necessary requirements for operator deployment.
- **Namespace Management:** Define and create the appropriate namespaces for the operator and its related resources prior to installation.
- **Helm Deployment:** Utilize Helm charts as the primary method to install and manage the operator lifecycle within your cluster.

=== Prerequisites for cluster deployment

Prerequisites for cluster deployment

Before deploying the `krkn-operator` into your environment, it is essential to prepare your Kubernetes cluster to ensure compatibility, security, and resource availability. Chaos engineering experiments, by design, stress the underlying infrastructure; therefore, a well-prepared foundation is critical for both the stability of your platform and the validity of your

chaos results.

Cluster Environment Requirements

To ensure the `krkn-operator` functions correctly, your environment must meet the following criteria:

- **Kubernetes Version:** Ensure your cluster is running a supported version of Kubernetes. Refer to the official Krkn release notes for the specific minimum version required for the operator version you intend to deploy.
- **Permissions:** You must have `cluster-admin` privileges or appropriate RBAC permissions to create Namespaces, ClusterRoles, and Custom Resource Definitions (CRDs).
- **Connectivity:** The operator must have network reachability to the API server of the target clusters. If you are utilizing multi-cluster execution, ensure that the communication paths between the management cluster and target clusters are open.
- **Resource Sizing:**
 - As noted in our best practices, applications often consume higher resource levels during reinitialization following a crash (e.g., replaying Write Ahead Logs in monitoring solutions like Prometheus).
 - Ensure your worker nodes have sufficient headroom in terms of CPU and Memory to accommodate these spikes during recovery phases. Failing to do so can lead to resource contention that impacts both your application and system-critical pods.

Access and Configuration Prerequisites

Before the installation process begins, ensure you have the following components configured:

Component	Requirement
CLI Access	<code>kubectl</code> or <code>oc</code> (for OpenShift environments) must be configured and authenticated against the target cluster.
Helm	The <code>helm</code> binary (version 3.x) is required for deploying the operator charts.
Registry Access	If your cluster resides in a restricted environment, ensure you have access to your private container registries. Configure image pull secrets in the target namespace beforehand to avoid <code>ImagePullBackOff</code> errors.
Namespace Strategy	Determine the dedicated namespace for the operator. While it can be installed cluster-wide, isolating the operator in a specific namespace (e.g., <code>krkn-chaos</code>) is recommended for security and observability.

Best Practices for Chaos Readiness

Beyond technical requirements, consider these operational prerequisites:

- **Node Sizing:** Review your node allocation strategies. Chaos experiments should target specific components to understand their behavior under stress. If your nodes are tightly

packed with production workloads, ensure that the "fast recovery logic" of your applications is tested prior to executing disruptive experiments.

- **Observability:** Confirm that your cluster has monitoring (e.g., Prometheus/Grafana) installed. The `krkn-operator` leverages PromQL-based metrics for the Resiliency Score feature. Without an active metrics pipeline, you will be unable to measure the impact of your experiments effectively.
- **Isolation:** Ensure that the target clusters are clearly identified. If using ACM/OCM integration for automatic discovery, ensure the credentials for those platforms are prepared and verified before attempting to register clusters within the operator interface.

Never execute chaos experiments on a production cluster without first validating the experiment workflows in a staging or development environment. Always monitor cluster health closely during the initial deployment phase to ensure the operator's own resource footprint does not interfere with existing workloads. ===# Prerequisites for cluster deployment

Prerequisites for cluster deployment

Before deploying the `krkn-operator` into your Kubernetes environment, it is essential to ensure that your cluster meets the following technical requirements. Proper preparation ensures that the operator functions correctly, especially when interacting with target clusters for chaos orchestration.

System Requirements

- **Kubernetes Version:** Ensure you are running a supported version of Kubernetes (refer to the specific operator release notes for the minimum version, typically the last three minor releases).
- **Compute Resources:** The operator and its managed components require sufficient headroom. As noted in our best practices, applications—including the chaos controller itself—may spike in resource usage during initialization or re-recovery cycles.
- **Memory:** Ensure your node pools are sized to accommodate the operator pod and any associated monitoring sidecars.
- **Monitoring:** If you plan to utilize the **Resiliency Score** feature, ensure that a Prometheus instance is reachable within the cluster to execute the required PromQL-based metrics queries.

Permissions and Access

The `krkn-operator` requires elevated privileges to monitor and manipulate cluster resources.

- **Cluster-Admin Role:** The operator requires sufficient RBAC permissions to create, update, and delete resources across the namespaces it manages.
- **Namespace Management:** You must decide whether to deploy the operator in a dedicated namespace or a shared one. We recommend a dedicated namespace (e.g., `krkn-system`) to isolate the operator's footprint and simplify security auditing.
- **Target Cluster Registration:** If you intend to use the **Multi-cluster Execution** feature, ensure your environment allows network connectivity between the operator cluster and the target clusters. While OCM/ACM integration is optional, it simplifies automatic discovery; otherwise, you must prepare the credentials for manual registration of target providers.

Network and Connectivity

- **Private Registry Access:** If your environment restricts access to public container registries, you must pre-configure the `krkn-operator` to pull images from your internal private registry. Ensure that the image pull secrets are available in the target namespace.
- **Egress Requirements:** The operator needs egress access to any external APIs or private registries required for image retrieval and, if applicable, reporting metrics to your centralized observability stack.

Best Practice: Planning for Resilience

When preparing your cluster deployment, keep the "Resiliency Score" methodology in mind:



Resource Sizing

An application pod might use significantly more resources during reinitialization after a chaos-induced crash. Ensure your cluster autoscaler is configured to handle these bursts so that the operator's activity does not lead to resource starvation for your production workloads.

Before proceeding to the installation phase, verify your current cluster capacity using:

```
kubectl get nodes  
kubectl describe nodes | grep -A 5 "Allocated resources"
```

Once these prerequisites are met, you are ready to proceed with the Helm chart deployment.
Prerequisites for cluster deployment

Prerequisites for cluster deployment

Before deploying the `krkn-operator` into your environment, it is essential to prepare your Kubernetes cluster to ensure compatibility, security, and operational stability. The operator acts as a central control plane for chaos orchestration, requiring sufficient resources and specific permissions to interact with target clusters and manage experiment lifecycles.

Cluster Environment Requirements

The `krkn-operator` is designed for Kubernetes-native chaos engineering. To ensure a successful installation, your environment must meet the following criteria:

- **Kubernetes Version:** Ensure your cluster is running a supported version of Kubernetes (refer to the specific operator release notes for minimum version compatibility).
- **Compute Resources:** The operator platform requires consistent resource availability. As noted in our best practices, monitor your node sizing carefully. Chaos experiments—and the operator itself—may experience spikes in resource consumption, particularly when generating telemetry or managing multiple concurrent experiment jobs.
- **Permissions:** You must have `cluster-admin` privileges (or equivalent rights) to install the operator, manage Custom Resource Definitions (CRDs), and configure necessary Roles/ClusterRoles.

Network and Connectivity

The `krkn-operator` excels in multi-cluster environments. To leverage its full capabilities, ensure:

- **Egress Connectivity:** The operator requires egress access to container registries to pull the `krkn-operator` image and related components. If your organization restricts public access, configure your **Private Registries** ahead of time.
- **Control Plane Access:** Ensure the operator has network visibility to the API servers of the target clusters it will manage.
- **Note:** While OCM/ACM integration is optional and simplifies automatic cluster discovery, the operator remains fully functional using manual registration of target cluster kubeconfigs.

Preparation Checklist

Before initiating the Helm installation, verify the following:

Component	Requirement
Namespace	Identify a dedicated namespace for the operator to ensure resource isolation.
Helm	Ensure helm (v3.x+) is installed and authenticated with your cluster.
Registry Access	If using a private registry, have your pull secrets ready.
Storage	Ensure the underlying storage class supports the persistence requirements of the operator's backend database (if applicable for experiment history).

Best Practices for Stability

Chaos engineering inherently pushes infrastructure to its limits. To ensure the operator does not become a bottleneck:

1. **Node Sizing:** Always account for "reinitialization spikes." When an application pod crashes during a chaos experiment, its recovery may require more memory (e.g., replaying write-ahead logs in Prometheus). Ensure your nodes are sized to handle these spikes so the `krkn-operator` remains stable.
2. **Recovery Logic:** Prior to deployment, verify that your target applications possess robust, fast recovery logic. This reduces the risk of cascading failures while the operator manages the experiments.
3. **Isolation:** Consider deploying the operator in a separate management cluster or a dedicated namespace with resource quotas (Limits and Requests) to prevent the operator from competing for resources with critical workloads.

Once these prerequisites are met, you are ready to proceed with the namespace creation and Helm-based installation of the `krkn-operator`.

Installing via Helm charts

Installing via Helm charts

The `krkn-operator` is distributed as a Helm chart, facilitating standardized deployments across Kubernetes and OpenShift environments. By leveraging Helm, you can manage the operator lifecycle, perform version rollouts, and customize configurations through `values.yaml` files.

Prerequisites

Before initiating the installation, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** Version 1.19 or higher, or OpenShift 4.x.
- **Helm:** Version 3.0 or higher must be installed and configured in your local environment.
- **Cluster Access:** `cluster-admin` privileges (or sufficient permissions to create namespaces and CRDs) are required to install the operator.

Installation Methods

The installation approach depends on how you intend to expose the operator's web interface for scenario orchestration.

Environment	Recommended Access Method
Kubernetes	Gateway API (preferred) or Ingress
OpenShift	Routes (native)

Quick Start Deployment

For a standard installation using default settings, you can deploy the latest version (v0.4.5-beta) directly from the project's OCI registry. This command automatically creates the necessary namespace and deploys the operator components.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace
```

Production Deployment with Custom Configuration

For production workloads, it is best practice to manage configuration via a custom `values.yaml` file. This allows you to define specific resource limits, replica counts, and exposure methods tailored to your cluster architecture.

1. Create a `values.yaml` file locally:

```
# values.yaml example
replicaCount: 1
image:
  repository: quay.io/krkn-chaos/krkn-operator
  tag: 0.4.5-beta
# Add environment-specific overrides here
```

2. Deploy the operator referencing your configuration file:

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  -f values.yaml
```

Verification

Once the Helm command completes, verify that the operator pods are running correctly:

```
kubectl get pods -n krkn-operator-system
```

Ensure the pods are in the `Running` state and that all containers have passed their readiness probes before attempting to configure chaos experiments.

Installing the Krkn-operator via Helm

The Krkn-operator is packaged as a Helm chart, facilitating standardized deployments across

Kubernetes and OpenShift environments. Using Helm allows for repeatable installations, version tracking, and configuration management through custom `values.yaml` files.

Prerequisites

Before initiating the installation, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** Version 1.19 or higher, or OpenShift 4.x.
- **Helm:** Version 3.0 or higher must be installed and configured to communicate with your cluster.
- **Permissions:** Cluster-admin privileges (or equivalent) are required to create namespaces and deploy the operator.

Installation Methods

The Krkn-operator can be deployed using a standard Quick Start method for rapid evaluation or a Production installation for specific ingress/gateway requirements.

Quick Start

For testing or demonstration purposes, you can deploy the latest version (v0.4.5-beta) using the default configuration. This command automatically creates the `krkn-operator-system` namespace.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace
```

Production Installation

In a production environment, you must account for external access to the operator's management interface. Choose the method based on your infrastructure:

- **OpenShift:** Utilize native OpenShift Routes.
- **Kubernetes:** Use Gateway API (recommended) or Ingress controllers.

To customize these settings, create a `values.yaml` file containing your specific networking configurations and install using the following command:

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  -f values.yaml
```

Namespace Management

Best practice dictates isolating the Krkn-operator within its own dedicated namespace. The `--create`

`-namespace` flag in the Helm commands above ensures that `krkn-operator-system` is created if it does not already exist.



Keeping the operator in a dedicated namespace prevents resource contention and simplifies the application of RBAC (Role-Based Access Control) policies, ensuring that chaos orchestration remains scoped correctly.

Troubleshooting Installation

- **Registry Access:** Ensure your cluster has network egress to `quay.io` to pull the OCI chart.
- **Chart Version:** Always verify the latest available version by checking the Krkn-chaos repository, as OCI-based charts are immutable and tied to specific tags.
- **Resource Conflicts:** If the installation fails, check for existing resources in the `krkn-operator-system` namespace using `kubectl get all -n krkn-operator-system` to identify potential naming collisions. # Installing via Helm charts

Installing via Helm charts

The `krkn-operator` is distributed as a Helm chart, providing a standardized and repeatable method for deploying the operator into your Kubernetes or OpenShift clusters. Using Helm allows for easy version tracking, dependency management, and configuration through `values.yaml` files.

Prerequisites

Before beginning the installation, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** Version 1.19+ or OpenShift 4.x.
- **Helm:** Version 3.0 or higher must be installed and configured to communicate with your cluster.
- **Registry Access:** Ensure your cluster can pull images from `quay.io/krkn-chaos/charts/krkn-operator`.

Installation Methods

There are two primary ways to install the Krkn-operator depending on your environment and traffic requirements.

Quick Start

For a standard installation using default settings, you can deploy the latest version directly using the OCI-based Helm chart. This will create the necessary namespace and deploy the operator components.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace
```

Production Installation with Custom Configuration

In production environments, you will typically need to configure external access to the operator. Depending on your cluster type, the recommended methods are:

- **OpenShift:** Utilize native **Routes**.
- **Kubernetes:** Utilize **Gateway API** (recommended) or **Ingress**.

To apply custom configurations, create a `values.yaml` file tailored to your infrastructure requirements and pass it during the installation:

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  -f values.yaml
```

Post-Installation Verification

After running the installation command, verify that the operator components are successfully deploying in the target namespace:

```
# Check the status of the operator pods
kubectl get pods -n krkn-operator-system

# Check the Helm release status
helm list -n krkn-operator-system
```



If the operator fails to start, ensure that your `values.yaml` configuration is compatible with the version `0.4.5-beta`. Refer to the chart's `README.md` for specific configuration parameters related to authentication and resource limits.

Namespace management and creation

Namespace Management and Creation

In the context of the Krkn-operator, namespaces serve as the primary logical boundary for both the operator's control plane and the scope of chaos experiments. Proper namespace management is critical to ensuring that the Krkn-operator remains isolated, secure, and performant within your Kubernetes environment.

Overview of Namespace Strategy

The Krkn-operator requires a dedicated namespace to host its controller, web interface, and associated backend services. By isolating the operator, you prevent accidental interference with production workloads and ensure that Role-Based Access Control (RBAC) policies can be scoped effectively.

Key considerations for namespace management include: * **Isolation:** Keeping the operator separate from application workloads. * **Resource Quotas:** Applying resource limits to the operator namespace to ensure the chaos orchestration platform does not consume excessive cluster resources. * **RBAC Scoping:** Defining **RoleBindings** that allow the operator to manage resources in target namespaces without granting cluster-wide administrative privileges.

Managing Namespaces for Krkn-operator

To manage namespaces effectively, follow these best practices for setup and configuration:

1. Creating the Operator Namespace

Before deploying the Krkn-operator, you must create a dedicated namespace. This acts as the home for the operator's pods.

```
kubectl create namespace krkn-operator-system
```

2. Labeling for Security and Integration

If your cluster employs security policies (such as Pod Security Admission), ensure the namespace is labeled correctly to allow the operator pods to run with the necessary privileges.

```
kubectl label namespace krkn-operator-system pod-security.kubernetes.io/enforce=privileged
```

Hands-on Lab: Namespace Provisioning

In this activity, you will prepare your cluster by creating the required namespace for the Krkn-operator.

Prerequisites

- A running Kubernetes cluster.
- **kubectl** configured with cluster-admin privileges.

Step 1: Create the Operator Namespace

Execute the following command to create the workspace for the Krkn-operator:

```
kubectl create namespace krkn-operator
```

Step 2: Verify Namespace Readiness

Ensure the namespace exists and is ready to accept resource deployments:

```
kubectl get namespace krkn-operator
```

Step 3: Verify Permissions

As per the permission model, ensure that your administrative user has access to this namespace. If you are integrating this with the Krkn-operator's web interface later, you will need to map this namespace to a specific user group.



If you intend to run "Namespace Failures" (as defined in our documentation context), be aware that the Krkn-operator will require **delete** permissions on the target namespaces you designate for experimentation. Ensure your **ClusterRole** definitions reflect these requirements.

Step 4: Cleanup

If you need to remove the operator and clean up the namespace, use the following command:

```
kubectl delete namespace krkn-operator
```



Deleting a namespace is destructive and will remove all custom resource definitions (CRDs), chaos scenario configurations, and logs stored within that namespace. Always back up your configuration files before proceeding with deletion.

== Namespace Management and Creation

In the context of the **krkn-operator**, effective namespace management is critical for operational security, resource isolation, and scope definition. By isolating chaos experiments within dedicated namespaces, administrators can prevent unintended "blast radius" expansion during testing.

== Namespace Strategy for Krkn-operator

The **krkn-operator** follows Kubernetes best practices by utilizing namespaces to segregate the operator's control plane from the workload clusters and the specific chaos scenarios being executed.

Before deploying the operator, ensure your target clusters have a designated namespace ready to host the controller and its associated resources.

=== Best Practices for Namespace Allocation

- * **Isolation:** Deploy the **krkn-operator** in a restricted namespace (e.g., **krkn-system**) to separate it from the applications it intends to disrupt.
- * **RBAC Alignment:** Since the operator requires permissions to modify objects in target namespaces (like deleting namespaces during a **Namespace Failure** scenario), use dedicated namespaces to apply the principle of least privilege.
- * **Resource Quotas:** When running high-intensity experiments, such as CPU or Memory Hogs, define resource quotas on the target namespaces to prevent the experiment from starving the underlying Kubernetes node.

== Hands-on Lab: Preparing Namespaces

In this lab, you will prepare the environment by creating a dedicated namespace for the **krkn-operator** and a target namespace for your chaos experiments.

Step 1: Create the Operator Namespace To begin, create a dedicated namespace to house the **krkn-operator** installation.

```
kubectl create namespace krkn-system
kubectl label namespace krkn-system name=krkn-operator
```

Step 2: Create a Target Namespace It is recommended to create a separate namespace for testing scenarios. This allows you to safely perform experiments, such as **Namespace Failures**, without affecting production workloads.

```
kubectl create namespace chaos-targets
```

Step 3: Verify Namespace Status Ensure that the namespaces are active and ready for the operator deployment.

```
kubectl get namespaces | grep -E 'krkn-system|chaos-targets'
```

== Managing Permissions per Namespace As outlined in the **Permission Model**, the Admin must define group access for these namespaces. When configuring users, ensure that the groups assigned to your users include the necessary permissions (**View**, **Run**, **Delete**) for the **chaos-targets** namespace specifically.

If you are performing a **Namespace Failure** experiment, ensure the user executing the scenario has sufficient **Delete** permissions on the target namespace; otherwise, the **krkn-operator** will be blocked by the Kubernetes API. ===# Namespace management and creation

Namespace Management and Creation

In Kubernetes, namespaces provide a mechanism for isolating groups of resources within a single cluster. For the **Krkn-operator**, proper namespace management is critical to ensure that chaos experiments are contained, permissions are scoped correctly, and cluster resources are organized effectively.

Why Namespaces Matter for Krkn-operator

When deploying the Krkn-operator, namespaces serve three primary functions:

1. **Isolation:** Keeping the operator and its associated chaos experiments separate from application workloads.
2. **Resource Quotas:** Preventing a chaos experiment (e.g., a CPU or Memory Hog) from consuming all resources in a production namespace.
3. **Access Control:** Allowing administrators to grant or restrict access to specific chaos scenarios

based on the namespace where they are deployed.

Creating the Operator Namespace

Before installing the Krkn-operator, you must define a dedicated namespace. This allows for cleaner cleanup procedures and targeted monitoring of the operator's health.

```
# Create a dedicated namespace for the Krkn-operator
kubectl create namespace krkn-operator-system

# Verify the namespace was created successfully
kubectl get namespace krkn-operator-system
```

Configuring Namespace Scope

When managing users and permissions (as defined in the [Permission Model](#)), administrators associate groups with specific clusters. However, you can refine this by applying Kubernetes RBAC (Role-Based Access Control) to limit the Krkn-operator's scope of influence to specific target namespaces.

Best Practices for Namespace Management

- **Principle of Least Privilege:** Do not grant the Krkn-operator cluster-admin permissions if it only needs to perform experiments within a specific namespace (e.g., `app-staging`).
- **Cleanup:** When deleting namespaces as part of a chaos experiment (e.g., the **Namespace Failure** scenario), ensure that the Krkn-operator has the necessary cluster-level permissions to manage those specific objects, while restricting its ability to touch critical infrastructure namespaces like `kube-system`.

Hands-on Lab: Managing Namespaces for Chaos

In this exercise, you will create a target namespace for your chaos experiments.

1. Step 1: Create a sandbox namespace

```
kubectl create namespace chaos-sandbox
```

2. Step 2: Verify Permissions

Ensure the Krkn-operator service account has permissions to interact with the `chaos-sandbox` namespace. If you are using the default Helm installation, verify the RoleBinding:

```
kubectl get rolebindings -n chaos-sandbox
```

3. Step 3: Orchestrate a Namespace Failure

Once your namespace is prepared, you can target it via the Krkn-operator interface or

configuration file to simulate a namespace deletion.



Ensure no mission-critical applications are running in a namespace tagged for deletion during a "Namespace Failure" experiment, as this operation will remove the namespace and all contained resources for the defined duration.

4. Step 4: Cleanup

After the experiment concludes and the namespace is restored, verify the objects:

```
kubectl get pods -n chaos-sandbox
```

Installation

Prerequisites for cluster deployment

Installing via Helm charts

Namespace management and creation

Installation

Prerequisites for cluster deployment

Installing via Helm charts

Namespace management and creation

Configuration and Management

Configuration and Management

The Configuration and Management module provides the tools required to govern user access, infrastructure settings, and chaos experiment execution within the Krkn-operator platform.

- **User and Access Control:** Administrators manage users and groups, assigning permissions to ensure secure access to clusters and operational features.
- **Infrastructure Setup:** Configure core platform components, including target providers, private registries, and cluster registration.
- **Chaos Experimentation:** Define and manage reusable configuration files for chaos experiments and orchestrate complex scenarios directly through the web interface.
- **Resource Administration:** Utilize file management capabilities to organize text-based configurations for consistent use across Chaos Studio workflows.

Configuration and Management

The Krkn-operator provides centralized control for platform administration and chaos orchestration. Key management capabilities include:

- **User and Permission Control:** Administrators manage access by creating user accounts, defining groups, and assigning cluster-specific permissions.
- **Infrastructure Setup:** Configure essential infrastructure components, such as target cluster registration and integration with target providers like ACM.
- **Resource Management:** Upload and maintain reusable configuration and text-based files for consistent chaos experiment execution.
- **Chaos Orchestration:** Define, trigger, and monitor chaos experiments directly through the web interface to validate system resilience.
- **Registry Configuration:** Manage private container registries and control their visibility across the platform.

Configuration and Management

The Krkn-operator provides centralized administrative control to manage users, permissions, and chaos workflows effectively. Key capabilities include:

- **User and Access Control:** Administrators can register clusters, create groups, and manage user permissions to ensure secure access to chaos infrastructure.
- **Infrastructure Configuration:** Facilitates the setup of target providers (such as ACM integration) and private registries to support diverse deployment environments.
- **Scenario Orchestration:** Provides a web-based interface for designing, configuring, and executing complex chaos experiments.

- **Resource Management:** Enables the upload and management of reusable configuration files, allowing for consistent application of settings across various chaos scenarios.

Managing users and permissions

Managing Users and Permissions

The Krkn-operator utilizes a robust, group-based access control model to ensure secure multi-tenant chaos engineering. By decoupling permissions from individual users and associating them with functional groups, administrators can maintain a scalable security posture across multiple Kubernetes clusters.

The Permission Model

The security architecture is built on a hierarchical structure where the **Group** acts as the primary container for access policies. Every user within the platform must be assigned to at least one group; a user without a group assignment will have no visibility or operational access to the platform.

Key Components

- **Groups:** Logical entities created by Administrators that define the scope of interaction. A group configuration specifies:
- **Cluster Accessibility:** Which target clusters are available to members.
- **Permissions:** The granular operations allowed (View, Run, Delete).
- **Registry Visibility:** Which image registries can be accessed by the chaos experiments.
- **Users:** Individuals who inherit all permissions associated with their assigned group.

Role Definitions

Permissions are tailored to specific personas within your organization:

Role	Responsibilities
Administrator	Manages the infrastructure, registers target clusters, creates groups, maps users to groups, and configures private registries and target providers (such as ACM).
User	Performs operational tasks based on group membership. This includes managing experiment jobs, accessing cluster terminals, executing chaos scenarios, and designing workflows in Chaos Studio.

Operational Capabilities

The scope of a user's activity is determined by the permissions granted to their group:

- **View and Manage:** Ability to observe and interact with existing chaos jobs (requires View/Delete permissions).
- **Terminal Access:** Ability to open a shell into a target cluster (requires Run permission).

- **Scenario Execution:** Ability to trigger chaos experiments on assigned clusters (requires Run permission).
- **Chaos Studio:** Ability to design, modify, and save complex chaos workflows (requires Run permission).
- **File Management:** Ability to upload and manage files, scoped strictly by the group's visibility settings.

Admin Configuration Workflow

To manage the environment effectively, administrators follow this standard lifecycle:

1. **Define the Group:** Create a group and designate the specific target clusters that the group will interact with.
2. **Assign Permissions:** Select the appropriate permission level (View, Run, Delete) to define the group's capability on the assigned clusters.
3. **Assign Users:** Add users to the group. Upon assignment, the user immediately inherits the group's security context.



To ensure the principle of least privilege, audit your group assignments regularly. A user's access is strictly defined by their group—if a user needs access to a new cluster, add that cluster to their existing group or assign them to an additional group with the required access.

Troubleshooting Permission Issues

If a user reports that they cannot see a cluster or execute a command, verify the following:

- **Group Membership:** Ensure the user is explicitly assigned to a group.
- **Group Policies:** Check if the target cluster is correctly mapped to the user's group.
- **Permission Level:** Confirm that the user has the 'Run' permission, as this is a prerequisite for executing scenarios and using terminal features.

Managing users and permissions

The **krkn-operator** follows a robust, group-based access control model designed to ensure that chaos engineering activities are performed securely and within defined operational boundaries. By decoupling permissions from individual users and attaching them to groups, administrators can maintain consistent security policies across large teams.

The Permission Model

Access in the **krkn-operator** is hierarchical. Every user must belong to at least one group; a user without a group assignment has no access to the platform's resources.

Group-Based Permissions

Administrators manage access by creating groups and defining a set of constraints for that group.

Each group configuration includes:

- **Accessible Clusters:** Defines which target Kubernetes clusters the group can impact.
- **Permissions:** Determines the scope of actions allowed (View, Run, Delete).
- **Registry Visibility:** Controls which container registries are accessible for pulling chaos experiment images.

Roles and Capabilities

The platform defines two primary roles, each with distinct scopes of responsibility:

Role	Capabilities
Admin	Full system control: registers/removes target clusters, creates/manages groups and users, configures private registries, and sets up target providers (e.g., ACM integration).
User	Operational access scoped by group membership: views and manages jobs, executes scenarios, designs workflows in Chaos Studio, and manages files within permitted boundaries.



Every user must be assigned to a group to interact with the system. Without group membership, the user interface will remain restricted, and API calls will be rejected with an unauthorized status.

Operational Scope for Users

When a user is assigned to a group, their operational capabilities are restricted by the permissions granted to that group:

- **View and Manage Jobs:** Depends on the View/Delete permissions defined for the group.
- **Cluster Terminal:** Requires the **Run** permission.
- **Scenario Execution:** Users can only run chaos scenarios on clusters explicitly assigned to their group.
- **Chaos Studio:** Designing and saving workflows requires the **Run** permission.
- **File Management:** Access to upload or manage configuration files is scoped by the visibility settings of the user's group.

Administrative Workflow

Administrators are responsible for maintaining the security posture of the **krkn-operator**. The lifecycle of permission management typically follows this flow:

1. **Define Groups:** Create a logical grouping (e.g., **qa-team**, **sre-platform**) that aligns with organizational structure.
2. **Assign Resources:** Map specific target clusters and registries to the created group.
3. **Assign Users:** Add users to the groups to grant them the corresponding level of access.

To maintain a principle of least privilege, always start by assigning only the **View** permission to new users. Elevate to **Run** or **Delete** only when a user requires the ability to execute experiments or manage infrastructure lifecycle. ===# Managing users and permissions

= Managing Users and Permissions

The Krkn-operator utilizes a robust, group-based access control model to ensure that chaos experiments are performed securely and within the boundaries of organizational policy. By decoupling permissions from individual users and assigning them to groups, administrators can maintain consistent security postures across multiple clusters.

== The Permission Model

Access in the Krkn-operator environment is governed by **Groups**. A user cannot perform any actions on the platform unless they are explicitly assigned to a group.

=== Key Components * **Users**: Individuals who interact with the Krkn-operator. Every user must belong to at least one group to gain access to clusters, registries, or features. * **Groups**: Logical containers that bundle access rights. When an Admin creates a group, they define: * **Cluster Accessibility**: Which target clusters the group can interact with. * **Permissions**: The level of access (View, Run, Delete) granted to the group. * **Registry Visibility**: Which container registries are accessible for fetching chaos experiment images.



=== Roles and Responsibilities

There are two primary roles defined within the Krkn-operator framework:

- **Administrator**: Possesses full access to the platform. Admins are responsible for the foundational infrastructure, including:
 - Registering and removing target clusters.
 - Creating and managing user accounts.
 - Defining group boundaries and assigning permissions.
 - Configuring infrastructure integrations (e.g., ACM).
- **User**: Operates within the scope defined by their group membership. Depending on their assigned permissions, they can:
 - **View and Manage Jobs**: Monitor the status and logs of chaos experiments.
 - **Run Scenarios**: Execute chaos experiments on assigned clusters.
 - **Chaos Studio**: Design and refine workflows (requires **Run** permission).
 - **Terminal Access**: Utilize the cluster terminal for troubleshooting (requires **Run** permission).

Every user must belong to a group. A user without a group assignment will have no access to the platform, ensuring a "secure by default" posture.

== Administrative Workflow

Administrators manage the environment through a hierarchical flow:

1. **Define Groups:** Identify the necessary access levels based on project or environment requirements (e.g., `prod-admins`, `qa-testers`).
2. **Assign Clusters:** Associate specific target clusters with the relevant group.
3. **Assign Permissions:** Grant specific capabilities:
 - **View:** Allows monitoring of existing jobs and cluster state.
 - **Run:** Allows execution of scenarios, workflow design, and terminal access.
 - **Delete:** Allows the removal of jobs or experiment artifacts.
4. **Create Users:** Provision accounts and map them to the established groups.

When planning your permission model, adhere to the principle of least privilege. Only grant **Run** permissions to users who strictly require the ability to initiate destructive chaos experiments.

== Summary of Access Levels

Capability	View	Run	Delete		:--	:--	:--	:--		Monitor Job Status	Yes	Yes	
Yes		Execute Chaos Scenarios	No	Yes	No		Design Workflows	No	Yes	No			
Access Cluster Terminal	No	Yes	No		Delete Experiments	No	No	Yes					

=== Configuring chaos experiments

Configuring Chaos Experiments

Chaos experiments are the fundamental unit of work within the `Krkn-operator`. They define the "fault" or "stress" to be injected into your Kubernetes environment. Configuring these experiments requires a clear understanding of the experiment definition, the target scope, and the desired steady-state hypothesis.

Anatomy of a Chaos Experiment

In the context of the `Krkn-operator`, a chaos experiment is typically managed through custom resources (CRs) that define the parameters of the disruption. When configuring these, focus on three primary pillars:

1. **Targeting:** Identifying the specific resources (Pods, Nodes, Deployments, or Network policies) that will be subjected to the fault.
2. **Disruption Parameters:** Defining the type of stress (e.g., CPU/Memory hogging, network latency, pod kills) and its intensity.
3. **Guardrails:** Defining constraints to ensure the chaos does not exceed the acceptable impact boundaries, potentially using the Krkn signaling feature to pause or stop if health checks fail.

Best Practices for Configuration

When drafting your experiment configurations, adhere to the following principles:

- **Define the Scope:** Avoid broad, cluster-wide impacts unless explicitly testing global resiliency. Use label selectors to pin experiments to specific subsets of your infrastructure.
- **Observe, Don't Just Break:** Always pair your experiment with observability tools. A chaos experiment without telemetry is simply a system outage.
- **Steady-State Hypothesis:** Before configuring the experiment, define what "normal" looks like. The experiment configuration should be designed to validate that the system returns to this state after the disruption ceases.
- **Environment Sensitivity:** As noted in our testing guide, experiments configured for staging may need different thresholds than those intended for production.

Hands-on Lab: Configuring a Basic Pod-Kill Experiment

This activity guides you through creating a simple chaos experiment configuration to terminate pods within a specific namespace.

Prerequisites

- A running Kubernetes cluster.
- `krkn-operator` installed and running.
- A target application deployed in a namespace (e.g., `my-app`).

Step 1: Create the Experiment Manifest

Create a file named `pod-kill-experiment.yaml`. This configuration instructs the operator to target a deployment by its label.

```
apiVersion: krkn.redhat.com/v1beta1
kind: ChaosExperiment
metadata:
  name: pod-killer-demo
  namespace: krkn-installer
spec:
  experiment:
    type: "pod-kill"
    labelSelector: "app=my-web-service"
    namespace: "my-app"
    duration: "60s"
    count: 2 # Kill 2 pods
```

Step 2: Apply the Configuration

Apply the manifest to your cluster using `kubectl`:

```
kubectl apply -f pod-kill-experiment.yaml
```

Step 3: Verify Execution

Monitor the status of your experiment using the custom resource status field:

```
kubectl get chaosexperiment pod-killer-demo -n krkn-installer -o yaml
```

Observe the `status` block. You should see the progression from `Pending` to `Running`, and finally `Completed`.

Step 4: Validate Impact

Check your target namespace to verify the pods were restarted:

```
kubectl get pods -n my-app  
# Look for the 'Age' column to confirm pod recreation
```

If the experiment does not trigger as expected, check the `krkn-operator` controller logs. Use `kubectl logs -l name=krkn-operator -n krkn-installer` to identify potential permission issues or invalid label selectors. ===# Configuring chaos experiments# Configuring chaos experiments

Configuring Chaos Experiments

Chaos experiments are the core units of work within the `krkn-operator`. They define the "fault" or "stress" to be injected into your Kubernetes environment. Configuring these experiments requires a clear understanding of the experiment lifecycle, scope, and the specific parameters required to simulate real-world failure scenarios effectively.

Anatomy of a Chaos Experiment

In the context of the `krkn-operator`, experiments are typically defined as Kubernetes Custom Resources (CRs). A well-configured experiment consists of three primary components:

- **Target Scope:** Defines the blast radius. This includes the namespaces, labels, or specific nodes/pods that will be subjected to the chaos.
- **Fault Definition:** Specifies the type of disruption (e.g., pod kill, network latency, memory stress, CPU hogging).
- **Observation/Signaling:** Configuration that coordinates the chaos run with other test jobs (e.g., Performance or Scalability testing).



Before executing an experiment, always define the scope clearly. As noted in our testing methodology, a narrow, well-defined scope prevents multiple infrastructure issues from masking one another during analysis.

Configuring Experiment Parameters

To configure an experiment, you will define a YAML manifest that the `krkn-operator` controller watches. Common configuration parameters include:

- `duration`: The time window the experiment should run.
- `iterations`: How many times the fault should be applied.
- `labelSelector`: A precise selector to ensure the chaos impacts only the intended workloads.
- `probability`: The likelihood of a specific instance being targeted (useful for partial cluster testing).

Integrating with Test Pipelines

When configuring experiments, consider the environment in which they execute:

1. **CI/CD Integration:** Configure your experiments to be automated. Use the `krkn-operator` signaling feature to start, stop, or pause scenarios based on the state of your ongoing performance or scalability test jobs.
2. **Continuous Chaos:** Treat chaos as a baseline. Configure experiments to run on a regular cadence to ensure that system health remains resilient as your software and infrastructure evolve.
3. **Observability:** Ensure that your experiment configuration includes monitoring hooks so that system reactions can be captured. The goal is to understand **how** the system responds rather than simply measuring a pass/fail state.

Hands-on Lab: Configuring a Basic Pod-Kill Experiment

In this activity, you will define a simple chaos experiment to terminate pods in a designated namespace.

Step 1: Create the Experiment Manifest

Create a file named `pod-kill-experiment.yaml`. This manifest instructs the operator to target a specific deployment.

```
apiVersion: krkn.redhat.com/v1beta1
kind: ChaosExperiment
metadata:
  name: sample-pod-kill
  namespace: krkn-hub
spec:
  experiment: "pod-kill"
  parameters:
    duration: "120s"
    labelSelector: "app=my-web-app"
    namespace: "production"
  signaling:
    enabled: true
```

Step 2: Apply the Configuration

Apply the configuration to your cluster. The `krkn-operator` will detect the resource and prepare the execution engine.

```
kubectl apply -f pod-kill-experiment.yaml -n krkn-hub
```

Step 3: Validate the Configuration

Verify that the operator has registered the experiment and it is ready for execution.

```
kubectl get chaosexperiments -n krkn-hub
```



If you are running this in a production-like environment, ensure that your `labelSelector` is highly specific to avoid accidentally impacting critical system components. Always verify your target list before triggering the execution via the web interface.

Troubleshooting Configuration Issues

- **Permission Denied:** Ensure the ServiceAccount assigned to the `krkn-operator` has the necessary RBAC roles to delete pods or modify network policies in the target namespace.
- **Orphaned Resources:** If an experiment is interrupted, check for remaining chaos pods. The operator is designed to clean these up, but manual intervention may be required if the operator pod itself is restarted during an experiment.

Orchestrating scenarios via the web interface

Orchestrating scenarios via the web interface

The Krkn-operator web interface provides a centralized control plane for chaos orchestration, moving beyond command-line execution to a visual, intuitive workflow management system. This interface allows operators to configure, execute, and monitor resilience tests across Kubernetes clusters seamlessly.

Core Interface Components

The web interface is designed to streamline the lifecycle of a chaos experiment through the following key modules:

- **Run Scenarios:** The primary gateway for executing single chaos scenarios. It allows for quick testing of specific failure injections without the overhead of complex workflow design.
- **Chaos Studio:** A drag-and-drop environment for designing sophisticated chaos experiments. It supports both serial and parallel execution, enabling the construction of complex failure graphs to simulate real-world production outages.
- **Jobs:** A real-time monitoring dashboard where you can inspect scenario execution progress, logs, and final results.
- **File Management:** A built-in repository for uploading and managing custom configuration files (YAML/JSON) required for specific experiments.
- **Cluster Terminal:** A read-only utility that allows you to explore target clusters, providing visibility into the state of your infrastructure during or after chaos execution.

Configuring Scenario Parameters

Before launching any scenario, you must define the operational parameters. These are categorized into three distinct levels:

1. **Mandatory:** Parameters required for the scenario to function (e.g., target namespace, duration). The operator will flag these as "must-configure" before the launch button is enabled.
2. **Optional:** Settings that provide fine-grained control over the experiment, such as specific label selectors, resource filters, or timing delays.
3. **Global:** Framework-level settings that govern the overall environment. This includes integration points like Prometheus for metrics, Elasticsearch for logging, and Cerberus for

status monitoring.

Executing and Monitoring

Once the parameters are defined, orchestration follows a simple two-step process:

1. **Launch:** Initiate the scenario or workflow via the web interface.
2. **Monitor:** You will be automatically redirected to the **Jobs** page. Here, you can watch the scenario's execution in real-time, inspect resource transitions, and verify if the resiliency score meets your organizational thresholds.

Hands-on Lab: Executing a Scenario

In this exercise, you will trigger a sample chaos scenario using the web interface.

Prerequisites

- Krkn-operator successfully deployed in your cluster.
- Access to the web interface URL.

Step 1: Accessing the Run Scenarios Module

1. Navigate to the Krkn-operator dashboard.
2. Select **Run Scenarios** from the sidebar.
3. Select a pre-loaded scenario (e.g., **pod-kill**) from the library.

Step 2: Configuring Parameters

1. In the configuration panel, ensure all **Mandatory** fields are filled.
2. Under the **Optional** tab, add a **label selector** to target only the **app=frontend** pods.
3. Verify that the **Global** settings are connected to your monitoring stack.

Step 3: Launch and Observe

1. Click the **Run** button.
2. Immediately observe the **Jobs** page.
3. Watch as the status transitions from **Pending** to **Running** and finally to **Completed**.
4. Click on the job name to view the detailed logs and the resiliency output.



If a job fails, use the **Cluster Terminal** to inspect the target resource logs to ensure the chaos injection reached the intended pod or node. ===# Orchestrating scenarios via the web interface

= Orchestrating scenarios via the web interface

The Krkn-operator web interface provides a centralized control plane for managing, designing, and executing chaos experiments. By abstracting the

underlying Kubernetes custom resources, the interface allows operators to focus on resiliency testing rather than manual YAML configuration.

== Understanding Scenario Orchestration Orchestration within the web interface is categorized into two primary modes:

- **Single Scenario Execution:** Ideal for targeted testing of a specific failure mode (e.g., pod kill or network latency).
- **Chaos Studio Workflows:** A visual drag-and-drop environment for designing complex, multi-stage resilience tests involving both serial and parallel execution.

== Configuration Parameters Before launching any scenario, you must configure the parameters associated with the chosen experiment. These are structured into three distinct tiers:

Category	Description
Mandatory	Required settings that must be defined before the "Run" button is enabled (e.g., target namespace, resource labels).
Optional	Fine-grained controls such as duration, intensity, or specific filters to refine the experiment scope.
Global	Framework-level configurations, including integration settings for Prometheus, Elasticsearch, or Cerberus to track resiliency metrics.

== Executing and Monitoring Once parameters are set, you can initiate the scenario. The workflow follows this standard lifecycle:

1. **Launch:** Initiate the execution via the Run action.
2. **Redirection:** The interface automatically navigates to the **Jobs** page.
3. **Real-time Monitoring:** Use the Jobs view to inspect logs, status transitions, and final outcome metrics for the scenario.

== Key Interface Features for Orchestration To manage your chaos program effectively, the interface provides several integrated tools:

- **Chaos Studio:** Use this visual builder to construct dependency graphs. You can chain scenarios so that a network partition (Scenario A) runs simultaneously with a node restart (Scenario B) to test compound failure scenarios.
- **Cluster Terminal:** A read-only utility that allows you to verify the state of your target clusters before and after chaos injection.
- **File Management:** A centralized repository for uploading and versioning your experiment configuration files, ensuring consistency across environments.

== Hands-on Lab: Executing a Scenario In this lab, you will orchestrate your first chaos experiment using the web interface.

=== Prerequisites * Krkn-operator deployed in your cluster. * Access to the web dashboard.

=== Step 1: Design a Basic Scenario 1. Navigate to **Run Scenarios** in the main navigation menu. 2. Select a standard scenario (e.g., **pod-delete**). 3. Fill in the **Mandatory** fields, such as **TARGET_NAMESPACE** and **LABEL_SELECTOR**.

=== Step 2: Configure Fine-Grained Controls 1. Expand the **Optional** settings section. 2. Adjust the **DURATION** parameter to control how long the chaos experiment persists. 3. Verify that your **Global** settings are pointed to your observability stack (e.g., Prometheus) to ensure the experiment metrics are captured.

=== Step 3: Execute and Observe 1. Click **Run**. 2. Observe the automatic transition to the **Jobs** page. 3. Monitor the status change from **Pending** to **Running** to **Completed**. 4. Verify the experiment results by checking the logs provided in the Job details view.

== [Configuration and Management](#)

=== [Managing users and permissions](#)

== [Configuration and Management](#)

=== [Managing users and permissions](#)

=== [Configuring chaos experiments](#)

=== [Orchestrating scenarios via the web interface](#)

== Lifecycle Management

Lifecycle Management

Lifecycle management ensures the ongoing reliability and operational integrity of the Krkn-operator within a Kubernetes environment. Key activities include:

- **Upgrades:** Performing seamless operator updates to incorporate new features and performance improvements.
 - **Version Control:** Managing releases and compatibility to ensure consistency across chaos experiment environments.
 - **Maintenance:** Monitoring operator health to ensure stable execution of chaos scenarios and continued adherence to defined service level indicators.
- Lifecycle Management

Lifecycle Management

Lifecycle management for the Krkn-operator encompasses the administrative tasks required to maintain a stable and current chaos engineering environment within your Kubernetes cluster.

- **Upgrades:** Performing seamless operator updates to ensure the latest features and stability patches are applied.
- **Versioning:** Managing release cycles to maintain compatibility with your cluster environment and experiment requirements.
- **Health Maintenance:** Monitoring operator performance and resource utilization to ensure consistent availability for orchestrating chaos scenarios.

=== Performing operator upgrades

Performing operator upgrades

Upgrading the **krkn-operator** is a critical administrative task to ensure your chaos engineering environment benefits from the latest features, security patches, and experiment definitions. As a Kubernetes-native tool, the operator lifecycle is managed primarily through Helm, allowing for seamless updates within your cluster.

Prerequisites for Upgrades

Before performing an upgrade, ensure the following conditions are met: * **Helm CLI:** Ensure you have the latest version of Helm installed locally. * **Cluster Access:** You must have **cluster-admin** or appropriate namespace-level permissions to modify the operator resources. * **Backup:** While the operator is stateless, ensure your custom chaos scenarios and experiment definitions are backed up or stored in version control (e.g., Git). * **Network Connectivity:** Ensure your cluster can reach the container registry at **quay.io** to pull the updated chart versions.

Performing the Upgrade

The **krkn-operator** is distributed as an OCI-compliant Helm chart. To upgrade the operator, you will point your local Helm client to the OCI registry and specify the desired version.

Step-by-Step Upgrade Procedure

1. **Verify Current Installation:** Check the currently installed version to confirm the release name and namespace: [source,bash] --- `helm list -n krkn-operator-system` ---
2. **Execute the Upgrade Command:** Use the **helm upgrade** command to pull the latest version from the OCI registry. Replace **0.4.5-beta** with your target version: [source,bash] --- `helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \ --version 0.4.5-beta \ --namespace krkn-operator-system` ---
3. **Validate the Upgrade:** After the command completes, verify that the operator pods have restarted and are running the new image: [source,bash] --- `kubectl get pods -n krkn-operator-system` ---

Troubleshooting Upgrade Issues

If the upgrade process encounters issues, follow these diagnostic steps:

- **Check Rollout Status:** If the operator fails to start, check the logs of the manager pod: [source,bash] ---- `kubectl logs -l control-plane=controller-manager -n krkn-operator-system` ----
- **Inspect Helm History:** If you need to revert to a previous working state, use the rollback feature: [source,bash] ---- `helm rollback krkn-operator <revision_number> -n krkn-operator-system` ----
- **Permission Errors:** If you receive "Access Denied" errors, verify that your service account or kubeconfig has the necessary permissions to update deployments and cluster roles within the `krkn-operator-system` namespace.

Best Practices

- **Version Pinning:** Always specify a `--version` flag to avoid accidental updates to unstable or incompatible releases.
- **Read Release Notes:** Before upgrading, consult the official release notes for any breaking changes in the Custom Resource Definitions (CRDs) or configuration schema.
- **Testing:** Perform upgrades in a non-production staging cluster before applying them to your production chaos orchestration environment.# Performing operator upgrades

Performing operator upgrades

Upgrading the Krkn-operator ensures that your chaos engineering framework benefits from the latest stability improvements, security patches, and new experiment scenarios. This section details the standard procedure for upgrading the operator using Helm.

Prerequisites

Before initiating an upgrade, ensure that:

- * You have access to the cluster where the Krkn-operator is deployed.
- * Your local `kubectl` and `helm` clients are configured to point to the correct cluster context.
- * You have reviewed the release notes for the target version to identify any breaking changes or required configuration migrations.

Upgrade Procedure

The Krkn-operator is managed via Helm charts. To upgrade the operator, you utilize the `helm upgrade` command, pointing to the specific OCI registry path.

Step-by-Step Upgrade

1. **Identify the Target Version:** Check the [Krkn-chaos repository](<https://quay.io/>)

[repository/krkn-chaos/charts/krkn-operator](#)) to identify the desired version tag.

2. **Execute the Upgrade:** Run the following command in your terminal, replacing the version number as needed:

```
helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-  
operator \  
  --version 0.4.5-beta \  
  --namespace krkn-operator-system
```

If you have customized values (e.g., specific resource limits or node selectors), ensure you include your custom `values.yaml` using the `-f` flag to prevent your local configurations from being overwritten: `helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator -f my-values.yaml --namespace krkn-operator-system`

Verifying the Upgrade

After running the upgrade command, verify that the operator has transitioned to the new version successfully:

1. **Check Deployment Status:**

```
kubectl rollout status deployment/krkn-operator-controller-manager -n krkn-  
operator-system
```

2. **Verify Versioning:** Describe the deployment or inspect the image tag to confirm the expected version is running:

```
kubectl get deployment krkn-operator-controller-manager -n krkn-operator-system  
-o yaml | grep image:
```

Troubleshooting

If the upgrade process stalls or the operator fails to start:

- **Rollback:** If the new version causes issues, you can revert to the previous release: `helm rollback krkn-operator -n krkn-operator-system`
- **Check Logs:** Inspect the controller logs for initialization errors: `kubectl logs -l control-plane=controller-manager -n krkn-operator-system`
- **CRD Compatibility:** Occasionally, new versions introduce updated Custom Resource Definitions (CRDs). Ensure that your `kubectl` version is sufficient to handle the CRD updates provided in the chart.

=== Handling versioning and releases

Handling versioning and releases

As the `krkn-operator` evolves to support new Kubernetes primitives and advanced chaos scenarios, maintaining a clear versioning strategy is essential for cluster stability. The operator follows standard software lifecycle practices to ensure that updates do not inadvertently trigger destructive chaos events or interrupt observability pipelines.

Understanding Versioning Strategy

The `krkn-operator` follows [Semantic Versioning (SemVer)](<https://semver.org/>), which uses the format `MAJOR.MINOR.PATCH` (e.g., `1.2.4`). This structure allows users to anticipate the impact of an upgrade:

- **MAJOR version (X.0.0):** Indicates breaking changes. This often involves changes to the Custom Resource Definitions (CRDs) or architectural shifts in how the operator interacts with the `krkn-lib`. Upgrading across major versions may require manual intervention or reconfiguration of existing chaos scenarios.
- **MINOR version (x.Y.0):** Introduces new features, such as new chaos scenarios or expanded support for cloud-provider-specific disruptions, while maintaining backward compatibility.
- **PATCH version (x.y.Z):** Focuses on bug fixes, performance optimizations for `krkn-lib` integrations, or security patches. These are generally safe to apply in production environments.

Release Lifecycle

Releases for the `krkn-operator` are managed through the GitHub repository ecosystem. When a new release is cut, the following artifacts are updated:

1. **Helm Charts:** The repository's Helm chart version is incremented to match the operator image tag.
2. **Container Images:** New images are pushed to the container registry, tagged with the specific release version and a `latest` tag (though pinning to specific versions is recommended for production).
3. **Changelog:** Each release includes a detailed changelog highlighting new features, deprecated items, and fixed issues.

Best Practices for Upgrades

Before performing an upgrade in a production-grade cluster, we recommend the following workflow:

- **Review the Changelog:** Always check the release notes for "Breaking Changes." If you are using custom CRDs, ensure they remain compatible with the new operator version.
- **Use a Staging Environment:** Test the upgrade in a non-production cluster where `krkn-demos` can be run to verify that chaos scenarios execute as expected.
- **Verify CRD Compatibility:** Since the operator relies on Kubernetes-native objects, verify if

the new release requires a `kubectl apply` of new or updated CRD manifests before updating the Helm deployment.

- **Monitor Operator Health:** Following an upgrade, observe the operator logs for initialization errors. Ensure the `krkn-lib` dependencies are correctly initialized and the operator is able to communicate with the Kubernetes API server.

Rollback Strategy

In the event that a new version introduces unexpected behavior, the operator can be reverted using Helm:

```
# List history of releases
helm history krkn-operator -n krkn-namespace

# Rollback to the previous stable release
helm rollback krkn-operator [REVISION_NUMBER] -n krkn-namespace
```

By maintaining strict version control and following the release lifecycle, teams can safely integrate Chaos Engineering into their continuous delivery pipelines while ensuring the `krkn-operator` remains a stable and predictable component of the infrastructure stack.

Handling versioning and releases

Handling versioning and releases

In the context of the `Krkn-operator`, managing versions and releases is critical to ensuring cluster stability while benefiting from the latest chaos engineering capabilities. As Krkn matures, understanding how the operator lifecycle aligns with the broader Krkn ecosystem (including `krkn-lib` and `Krkn-hub`) ensures that your chaos experiments remain predictable and repeatable.

Understanding Krkn Versioning

The Krkn ecosystem follows a structured versioning strategy designed to maintain compatibility across the dependency chain. Because the operator relies on underlying Python functions provided by `krkn-lib`, releases are synchronized to prevent breaking changes during runtime.

- **SemVer Compliance:** Releases follow Semantic Versioning (MAJOR.MINOR.PATCH).
- **MAJOR:** Significant changes to the operator API, CRDs (Custom Resource Definitions), or major architectural shifts.
- **MINOR:** New chaos scenarios or features added to the Krkn ecosystem.
- **PATCH:** Security updates, bug fixes, or stability improvements for existing chaos experiments.

Release Strategy and Compatibility

When planning an upgrade, it is important to consider the relationship between the operator

and the core Krkn engine.

Before upgrading the **Krkn-operator**, always verify the version of the **krkn-lib** dependency specified in the release notes. While the operator manages the lifecycle, the core logic resides in the engine; mismatches can result in failed telemetry data or execution errors.

Best Practices for Releases

1. **Check the Release Notes:** Always review the GitHub releases page for the **krkn-operator** repository to identify breaking changes in CRD schemas.
2. **Telemetry Data Integrity:** Krkn outputs telemetry data at the end of runs. When moving between versions, ensure that your post-processing scripts are compatible with any schema changes defined in the new release's **krkn-lib** models.
3. **Environment Isolation:** If you are using **Krkn-hub** in your CI/CD pipelines, ensure that your environment variables are pinned to specific operator versions to prevent unexpected chaos behaviors during automated testing.

Identifying Version Information

You can retrieve the current version of your deployed operator using **kubectl**. This is useful for auditing and ensuring your environment is running the expected release.

```
# Check the image tag currently running in the operator deployment
kubectl get deployment krkn-operator-controller-manager -n krkn-operator-system \
-o jsonpath='{.spec.template.spec.containers[0].image}'
```

Upgrade Considerations

When performing an upgrade, the operator lifecycle manager (or manual Helm upgrade) handles the migration of existing resources. However, you should observe the following:

- **Stateful Experiments:** If a chaos experiment is currently running, avoid performing an operator upgrade. The interruption of the operator's control loop during an active scenario may result in orphaned chaos resources (e.g., pods that were meant to be killed or network latency that was meant to be cleared).
- **Backup Configs:** Use the File Management capabilities of the operator to export your existing configuration files before applying a version upgrade. This ensures you can revert to known-good configurations if the new version introduces unexpected behaviors.

By adhering to these versioning and release practices, you maintain a robust resilience testing pipeline that is both stable and continuously improving.

=== Maintaining operator health

Maintaining Operator Health

Maintaining the health of the `krkn-operator` is critical to ensuring that chaos experiments are executed reliably without compromising the stability of the management plane. A healthy operator ensures that chaos scenarios are orchestrated correctly, resources are cleaned up post-experiment, and the system remains observable.

Health Monitoring Best Practices

To ensure the `krkn-operator` functions within expected parameters, consider the following maintenance strategies:

- **Resource Monitoring:** Monitor the memory and CPU utilization of the `krkn-controller-manager` pods. Excessive resource consumption may indicate memory leaks or an unoptimized reconciliation loop.
- **Log Analysis:** Regularly inspect operator logs for errors related to API connectivity or permission denials. Using `kubectl logs -n krkn-namespace deployment/krkn-controller-manager` is standard practice.
- **Condition Tracking:** Leverage Kubernetes status conditions. Ensure the operator reports `Ready` and `Available` statuses. If the operator enters a `CrashLoopBackOff`, investigate the configuration map for syntax errors or invalid experiment parameters.
- **Event Auditing:** Watch for Kubernetes events within the namespace where the operator is deployed to catch early warnings regarding resource quota exhaustion or RBAC failures.

Ensuring System Stability

The stability of the operator is intrinsically linked to the underlying cluster health. Apply the following administration standards:

Machine Health Checks: Always deploy machine health checks with appropriate conditions to remediate unhealthy nodes. This ensures that even if a chaos experiment targets infrastructure, the cluster maintains enough capacity to run without total downtime, preventing the operator from struggling to reconcile state on degraded hardware.

Proactive Maintenance Steps

1. **Reconciliation Audits:** Periodically check the number of active chaos resources. An accumulation of orphaned `ChaosExperiment` custom resources (CRs) can bloat the etcd database and slow down the controller reconciliation loop.
2. **RBAC Verification:** If you modify service accounts or cluster roles, ensure the operator retains the necessary permissions to interact with target resources. Use `kubectl auth can-i` to verify that the operator's service account has the required verb access (e.g., `list`, `patch`, `delete`) to the target namespaces.
3. **Dependency Updates:** Periodically verify that the underlying Kubernetes API versions supported by the operator are compatible with your current cluster version.

Troubleshooting Health Issues

If the operator exhibits unexpected behavior, follow this diagnostic workflow:

1. Check Pod Status:

```
kubectl get pods -n krkn-operator-system
```

2. Inspect Operator Logs: Look for stack traces or connection timeouts.

```
kubectl logs -n krkn-operator-system -l control-plane=controller-manager  
--tail=100
```

3. Verify Resource Limits: Ensure that the `LimitRanges` and `ResourceQuotas` in the namespace do not restrict the operator from spawning the necessary pods to execute chaos experiments.

By integrating these maintenance tasks into your standard platform administration routine, you ensure that the `krkn-operator` remains a resilient tool for validating your service level indicators (SLIs) and key performance indicators (KPIs) under duress. # Maintaining operator health

Maintaining Operator Health

Maintaining the health of the `krkn-operator` is critical for ensuring that chaos experiments are executed reliably and that the operator does not become a bottleneck or a point of failure within your Kubernetes cluster. A healthy operator ensures that chaos scenarios are injected gracefully and that the reconciliation loop functions as expected.

Monitoring Operator Health

To maintain the operational integrity of the `krkn-operator`, administrators should focus on three primary pillars: observability, resource management, and state verification.

1. Observability and Metrics

The `krkn-operator` exposes Prometheus-formatted metrics that are essential for health monitoring. Use these metrics to create dashboards in Grafana or alerts in Alertmanager:

- * **Reconciliation Errors:** Monitor the `controller_runtime_reconcile_errors_total` metric. A spike here indicates that the operator is struggling to reconcile the state of chaos scenarios against the cluster.
- * **Work Queue Depth:** Ensure the work queue length remains stable. A growing queue suggests the operator is overloaded with pending tasks.
- * **Pod Readiness:** Use standard Kubernetes probes to track the operator pod's status. If the operator enters a `CrashLoopBackOff`, check the logs using:

+

```
kubectl logs -n krkn-namespace deployment/krkn-operator-controller-manager
```

2. Resource Management

Chaos orchestration can be resource-intensive depending on the frequency and scale of your experiments. * **Resource Limits:** Ensure the `krkn-operator` deployment has appropriate CPU and memory requests/limits defined in its Helm values. * **Node Health Checks:** As per best practices, deploy Machine Health Checks within your cluster. If the node running the `krkn-operator` becomes unhealthy, the Cluster API or machine management system should remediate the node to ensure the operator is rescheduled, preventing downtime in your chaos orchestration pipeline.

3. Log Aggregation and Auditing

Regularly audit the operator logs to identify potential issues before they impact production environments. Look for: * **Permission Denied Errors:** Often caused by stale `ClusterRoleBinding` or `ServiceAccount` misconfigurations. * **Timeout Exceptions:** Verify connectivity between the operator and the Kubernetes API server.

Proactive Maintenance Strategies

To keep the operator in a stable state, follow these administrative routines:

- **Cleanup of Completed Scenarios:** Ensure that completed or failed chaos experiment CRDs (Custom Resource Definitions) are cleaned up to prevent etcd bloating.
- **Regular Version Alignment:** Keep the operator version aligned with your Kubernetes minor version. Using outdated operators with newer K8s APIs can lead to silent failures in experiment execution.
- **Validation of Service Level Indicators (SLIs):** Periodically verify that your chaos experiments are correctly evaluating your application's SLIs. If an experiment fails to report accurate status, investigate the `Status` field within the chaos CRD.

When failures occur in production, they are costly. Always test your chaos scenarios in a staging environment before deploying the `krkn-operator` to production clusters to ensure that the orchestration logic aligns with your resilience expectations.

Troubleshooting Common Health Issues

Symptom	Probable Cause	Recommended Action
High Reconciliation Time	Excessive CRDs in namespace	Cleanup old experiment objects.
Operator pod flapping	Resource constraints (OOMKilled)	Increase memory limits in <code>values.yaml</code> .

Experiments not triggering	RBAC configuration mismatch	Validate <code>ClusterRole</code> permissions for the operator ServiceAccount.
----------------------------	-----------------------------	--

By integrating these health checks into your platform administration routine, you ensure that the `krkn-operator` remains a reliable tool for testing system resilience and identifying weaknesses before they impact end-users.

== [Lifecycle Management](#)

=== [Performing operator upgrades](#)

=== [Handling versioning and releases](#)

=== [Maintaining operator health](#)

== Hands-on Lab

Hands-on Lab

The hands-on lab provides a practical environment to apply Krkn-operator concepts through real-world scenarios. It focuses on the following key activities:

- **Deployment:** Installing the Krkn-operator within a Kubernetes cluster to establish the testing environment.
- **Experimentation:** Executing sample chaos experiments to observe system behavior and validate resilience.
- **Maintenance:** Performing operator upgrades to manage versioning and ensure the platform remains up-to-date with the latest features.# Hands-on Lab

Hands-on Lab

The Hands-on Lab provides a practical environment to master Krkn-operator operations through direct application. Participants gain experience in:

- **Deployment:** Installing the Krkn-operator within a live Kubernetes cluster.
- **Execution:** Running sample chaos experiments to observe system behavior and platform functionality.
- **Maintenance:** Performing operator upgrades to manage lifecycle and versioning requirements effectively.

=== Deploying the Krkn-operator in a Kubernetes cluster

Deploying the Krkn-operator in a Kubernetes cluster

This section provides the procedure to deploy the Krkn-operator within your Kubernetes environment. The Krkn-operator acts as a centralized control plane for managing your chaos engineering experiments, allowing you to orchestrate scenarios through a unified web

interface rather than managing manual executions.

Prerequisites

Before deploying the Krkn-operator, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** A functional cluster (Kubernetes or OpenShift).
- **Helm:** Ensure Helm 3.x is installed on your local machine to manage the deployment.
- **External Access:** Decide on your access strategy. For Kubernetes environments, it is recommended to use the **Gateway API** or **Ingress** to expose the web interface.
- **Permissions:** You must have `cluster-admin` privileges to create namespaces and manage Custom Resource Definitions (CRDs) required by the operator.

Installation Procedure

The Krkn-operator is distributed via OCI-compliant Helm charts. Use the following steps to deploy the operator into your cluster.

Step 1: Create the Namespace

It is best practice to isolate the operator in its own namespace to maintain cluster hygiene.

```
kubectl create namespace krkn-operator-system
```

Step 2: Prepare the Configuration

While you can use default settings, a production-grade deployment typically requires a `values.yaml` file to define your specific ingress controller settings and persistent storage requirements.

Create a file named `values.yaml` and configure your parameters. For example, to enable OCM/ACM integration for automatic cluster discovery, include the following:

```
acm:
  enabled: true
# Add additional configuration for ingress or resource limits here
```

Step 3: Deploy via Helm

Execute the following command to install the Krkn-operator. This command pulls the chart directly from the official Krkn repository.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
```

```
-f values.yaml
```

If you are not using a `values.yaml` file, you can pass parameters directly via the command line. For instance, to enable ACM integration without a file: `--set acm.enabled=true`

Verification

Once the Helm installation completes, verify that the operator components are running correctly by checking the status of the pods in the `krkn-operator-system` namespace:

```
kubectl get pods -n krkn-operator-system
```

Ensure all pods are in the `Running` or `Completed` state. If the pods are failing to start, inspect the logs for configuration errors:

```
kubectl logs -l app.kubernetes.io/name=krkn-operator -n krkn-operator-system
```

Post-Deployment: Accessing the Interface

After the pods are stable, retrieve the address for the web interface based on the Ingress or Gateway API configuration provided in your `values.yaml`. You can then navigate to the provided URL to begin managing your chaos experiments. # Deploying the Krkn-operator in a Kubernetes cluster

Deploying the Krkn-operator in a Kubernetes cluster

This guide walks you through the process of deploying the Krkn-operator into your Kubernetes cluster using Helm. The Krkn-operator serves as the central orchestration engine for your chaos engineering experiments, providing a unified web interface for management and execution.

Prerequisites

Before beginning the installation, ensure your environment meets the following requirements:

- **Kubernetes Cluster:** A running Kubernetes cluster (version 1.25+ recommended).
- **Helm:** The Helm CLI tool installed on your local machine.
- **Connectivity:** Access to the `quay.io` container registry to pull the Krkn-operator images.
- **Permissions:** Cluster-admin privileges to create namespaces and manage Custom Resource Definitions (CRDs).

Deployment Strategies

Depending on your cluster environment and requirements, you can choose between a

standard deployment or an environment-specific integration.

1. Standard Deployment

For most standard Kubernetes environments, you will use the Helm chart provided in the OCI registry. This method creates a dedicated namespace and deploys the operator with default configurations.

2. ACM/OCM Integration

If you are managing multiple clusters via Open Cluster Management (OCM) or Red Hat Advanced Cluster Management (ACM), you can enable automatic cluster discovery by passing specific parameters during the Helm installation.

Hands-on Lab: Deploying Krkn-operator

Follow these steps to deploy the operator into your cluster.

Step 1: Prepare your Values File

Create a `values.yaml` file to customize your deployment. This allows you to define ingress controllers or specific resource constraints.

```
# values.yaml example
replicaCount: 1
# Configure ingress/gateway settings here
ingress:
  enabled: true
```

Step 2: Install the Operator

Execute the following command to install the Krkn-operator. This command will create the `krkn-operator-system` namespace automatically.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  -f values.yaml
```

Step 3: Deploying with ACM/OCM Enabled (Optional)

If your environment utilizes Open Cluster Management, perform the installation with the integration flag enabled:

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --integration-enabled
```

```
--create-namespace \  
--set acm.enabled=true
```

Step 4: Verification

Confirm that the operator pods are running correctly:

```
kubectl get pods -n krkn-operator-system
```

Expected output should show the operator pod in a **Running** state:

NAME	READY	STATUS	RESTARTS	AGE
krkn-operator-controller-manager-xxxx	1/1	Running	0	2m

Troubleshooting

- **Image Pull Errors:** Ensure your cluster has outbound network access to quay.io.
- **Namespace Stuck in Terminating:** If you are re-installing, ensure no orphaned CRDs from previous installations remain in the cluster.
- **Permission Denied:** Verify your `kubeconfig` context is set to a cluster-admin user.

=== Executing a sample chaos experiment

Executing a sample chaos experiment

This section guides you through the execution of a baseline chaos experiment using the **Krkn-operator**. We will trigger a pod-failure scenario to observe how your application and cluster infrastructure respond to unexpected disruptions.

Prerequisites

Before executing the experiment, ensure:

- * The **Krkn-operator** is successfully deployed in your cluster.
- * You have access to the Kubernetes namespace where the target application resides.
- * The **krkn** CLI tool or the operator's Custom Resource Definition (CRD) interface is configured.

Understanding the Experiment Workflow

Chaos experiments in the **Krkn-operator** are defined as Kubernetes Custom Resources (CRs). When you apply a chaos CR, the operator reconciles the state, coordinates the execution of the fault, and monitors the target resources.

Hands-on Lab: Executing a Pod-Failure Experiment

In this exercise, we will induce a controlled pod-failure within a specific deployment to test the resilience of your service.

Step 1: Create the Chaos Experiment Manifest

Create a file named `pod-failure-experiment.yaml`. This manifest instructs the operator to terminate pods matching a specific label.

```
apiVersion: krkn.redhat.com/v1beta1
kind: ChaosExperiment
metadata:
  name: sample-pod-failure
  namespace: krkn-operator-system
spec:
  experiment_type: "pod-failure"
  target_namespace: "my-app-namespace"
  label_selector: "app=frontend"
  duration: "60s"
  instances: 1
```

Step 2: Trigger the Experiment

Apply the manifest to your cluster to initiate the chaos orchestration.

```
kubectl apply -f pod-failure-experiment.yaml
```

Step 3: Monitor Execution

Use the following commands to observe the operator's progress and the state of your target application:

1. Check Operator logs:

```
kubectl logs -f deployment/krkn-operator-controller-manager -n krkn-operator-system
```

2. Observe the target pods:

```
kubectl get pods -n my-app-namespace -w
```

Step 4: Analyze Resilience

Once the duration defined in the YAML has elapsed, the operator will stop the experiment. Review the results:

- **Observation:** Did the application deployment automatically spin up a new pod?
- **Monitoring:** Check your observability dashboard (e.g., Prometheus/Grafana) to see if latency spiked or if health checks failed during the pod termination.
- **Learning:** Remember that the goal is not to "pass" or "fail" the test, but to understand if the

system recovers gracefully without manual intervention.

If you have configured the Kraken **Signaling feature**, you can use it here to pause other performance or scalability tests running in the background until the pod-failure experiment completes, ensuring that you isolate the chaos impact from other telemetry data.

Troubleshooting

If the experiment fails to start: * Verify that the `target_namespace` and `label_selector` match active resources in your cluster. * Ensure the service account used by the `Krkn-operator` has sufficient RBAC permissions to delete pods in the target namespace. * Check the status of the `ChaosExperiment` CR: `kubectl describe chaosexperiment sample-pod-failure -n krkn-operator-system`.# Executing a sample chaos experiment

Executing a sample chaos experiment

This section guides you through the execution of a fundamental chaos experiment using the `Krkn-operator`. We will focus on a standard pod-delete scenario, which is the cornerstone of verifying service resilience in a Kubernetes environment.

Prerequisites

Before starting this experiment, ensure that: * The `krkn-operator` is successfully deployed in your cluster. * You have `kubectl` access configured for the target namespace. * You have identified a non-critical application or a "test" deployment to target for the chaos injection.

Understanding the Experiment Structure

A chaos experiment in `Krkn` is defined by a `ChaosScenario` custom resource (CR). This resource dictates: * **The Fault Type:** (e.g., pod-delete, node-cpu-hog, network-latency). * **The Blast Radius:** Defined by labels, namespaces, or specific resource names. * **The Duration:** How long the chaos persists or how many iterations to perform.

Hands-on Lab: Executing a Pod-Delete Scenario

Follow these steps to inject a pod-deletion fault into your environment.

Step 1: Create the Scenario Manifest

Create a file named `pod-delete-scenario.yaml`. This manifest instructs the operator to terminate pods matching a specific label.

```
apiVersion: krkn.chaos/v1beta1
kind: ChaosScenario
metadata:
  name: sample-pod-delete
  namespace: krkn-operator
spec:
```

```
scenario: pod-delete
duration: 60s
parameters:
  label_selector: "app=demo-app"
  namespace: "default"
  grace_period: 0
```

Step 2: Apply the Scenario

Apply the manifest to the cluster to trigger the operator's execution logic.

```
kubectl apply -f pod-delete-scenario.yaml
```

Step 3: Observe the Impact

Once applied, the Krkn-operator will reconcile the state and begin the experiment. Use the following commands to monitor the behavior:

1. Check Operator Logs:

```
kubectl logs -f deployment/krkn-operator -n krkn-operator
```

2. Watch the Target Pods:

```
kubectl get pods -n default -w
```

Observation: You should see the pods with the label `app=demo-app` terminating and being recreated by their respective ReplicaSet.

Step 4: Verify Resilience

After the duration specified (60s), the chaos will subside. Evaluate the system recovery: * Did the application recover automatically? * Did the monitoring tools trigger alerts during the outage? * Check your observability dashboard to see the correlation between the pod-delete timestamps and your service metrics.

Learning over Passing: As noted in our best practices, do not focus on whether the test "passes." Focus on the system's reaction. If the pod-delete causes a cascade of failures, document the dependency chain—this is the primary goal of Chaos Engineering.

Troubleshooting

If the experiment does not trigger as expected: * **Check RBAC:** Ensure the Krkn-operator service account has sufficient permissions (ClusterRole/Role) to delete pods in the target namespace. * **Validate Selectors:** Ensure the `label_selector` in your CR matches the actual

labels on the target pods. * **Events:** Check Kubernetes events in the `krkn-operator` namespace for any rejection errors: `kubectl get events -n krkn-operator`.

=== Upgrading the operator to a newer version

Upgrading the Krkn-operator

As your chaos engineering requirements evolve, keeping the `krkn-operator` up to date ensures you have access to the latest chaos scenarios, performance improvements, and security patches. This section guides you through the process of upgrading the operator within your Kubernetes environment.

Prerequisites

- **Helm CLI:** Ensure you have the latest version of Helm installed.
- **Cluster Access:** You must have `cluster-admin` or appropriate permissions to modify resources within the `krkn-operator-system` namespace.
- **Version Verification:** Identify the target version you intend to upgrade to by checking the [Krkn-chaos repository](<https://github.com/krkn-chaos/krkn-operator>) or the OCI registry.

Upgrade Procedure

Upgrading the `krkn-operator` is a straightforward process using the Helm CLI. Since the operator is distributed via OCI-compliant charts, you can perform an in-place upgrade without removing the existing installation.

Before performing an upgrade in a production-like environment, it is recommended to verify the release notes of the target version to identify any potential breaking changes in CRD (Custom Resource Definition) schemas or configuration parameters.

Step-by-Step Upgrade

1. **Update your local Helm repositories** (if applicable) to ensure you are referencing the latest metadata.
2. **Execute the upgrade command** using the `helm upgrade` directive. Ensure you point to the same namespace used during the initial installation.

```
helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system
```

- `--version`: Specifies the target release version.
- `--namespace`: Must match the namespace where the operator is currently deployed.

Verifying the Upgrade

After executing the upgrade command, verify that the operator pods have rolled out successfully and are running the new image version.

1. Check the status of the release:

```
helm status krkn-operator -n krkn-operator-system
```

2. Monitor the operator pod rollout:

```
kubectl rollout status deployment/krkn-operator-controller-manager -n krkn-operator-system
```

Troubleshooting Common Issues

- **Namespace Mismatch:** If you receive a "release not found" error, verify that the namespace provided in the **upgrade** command matches the namespace where the operator was originally installed.
- **CRD Conflicts:** If the upgrade fails due to CRD changes, ensure that you have the necessary cluster-level permissions to update Kubernetes custom resources.
- **Image Pull Errors:** If the pods are stuck in **ImagePullBackOff**, ensure your cluster has network connectivity to **quay.io** and that the OCI registry remains accessible. # Upgrading the operator to a newer version

Upgrading the Krkn-operator

As part of the lifecycle management of the Krkn-operator, periodic upgrades are necessary to leverage new chaos scenarios, performance improvements, and security patches. This section guides you through the process of upgrading your existing installation using Helm.

Prerequisites

Before initiating an upgrade, ensure that:

- * You have **helm** installed and configured to access your Kubernetes cluster.
- * You have identified the target version (e.g., **0.4.5-beta**).
- * Your **kubectl** context is correctly set to the cluster where the operator is deployed.

Upgrade Process

Upgrading the Krkn-operator involves pointing your Helm release to the desired OCI artifact version. Because the operator state is managed via Custom Resource Definitions (CRDs), Helm handles the transition of the deployment and associated controller logic.

Step-by-Step Upgrade Procedure

To upgrade the operator to a newer version, follow these steps:

```
helm upgrade krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system
```

Note: Ensure the `--namespace` flag matches the namespace where your operator is currently installed.

Verification

After executing the upgrade command, verify that the operator has successfully rolled over to the new version:

1. Check the deployment status:

```
kubectl rollout status deployment/krkn-operator-controller-manager -n krkn-
operator-system
```

2. Verify the pod image version:

```
kubectl get pods -n krkn-operator-system -o
jsonpath='{.items[0].spec.containers[0].image}'
```

Troubleshooting Upgrade Issues

If the operator fails to come up after an upgrade:

- **Version Compatibility:** Verify that the OCI tag `0.4.5-beta` exists in the repository.
- **CRD Conflicts:** Occasionally, new versions introduce schema changes. If the controller fails to start, check the logs of the manager pod:

```
kubectl logs -l control-plane=controller-manager -n krkn-operator-system
```

- **Namespace Constraints:** Ensure the `--namespace` provided during the upgrade is identical to the one used during the initial installation to avoid split-brain scenarios or duplicate deployments.

== [Hands-on Lab](#)

=== [Deploying the Krkn-operator in a Kubernetes cluster](#)

=== [Executing a sample chaos experiment](#)

=== [Upgrading the operator to a newer version](#)

== [Appendix](#)

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)